

Joint simulation of heat wave impacts on leaves, grasshoppers and lizards

Michael Kearney

2025-02-12

Overview

Installing and loading packages

The first step is to install and load the three relevant packages.

```
library(devtools)
install.packages("plantecophys")
install_github("ilyamaclean/microclima") # install the microclima package
install_github("mrke/NicheMapR") # install NicheMapR
# where to put the global climate dataset
climate_data_path <- "put your path here"
# download and unpack the global climate data
get.global.climate(folder = climate_data_path)
```

```
library(NicheMapR)
library(tealeaves)
library(plantecophys)
```

Microclimate simulation

Next we simulate a microclimate with the ‘micro_silo’ function of NicheMapR. This function uses historical weather data from 1889 to yesterday <https://www.longpaddock.qld.gov.au/silo/>.

Here we run it for a site in the Australian arid zone, Calperum Station. The model calculates microclimate at a user-specified height ‘Usrhyt’ which in this example is 1 cm. It produces air temperature, wind speed and humidity estimates at this height and at the reference height for the forcing data, in this case 2 m. The ‘get_profile’ function allows estimates of air temperature, wind speed and humidity at any other height between the ground and the reference height. The simulation is run for two shade scenarios, a minimum shade (default 0%) and a maximum shade (default 90%) to provide a thermoregulating organism with a spectrum of shade options. Two simulations are run here, with different maximum shade levels, the low max shade scenario being used to simulate leaf dieoff.

Set location, date and some parameters (most left at default).

```
# simulate a microclimate
loc <- c(140.710701228588, -34.0432449004568) # longitude, latitude, decimal degrees
Usrhyt <- 0.01 # leaf height, m
dstart <- "01/01/2018"
```

```

dfinish <- "31/12/2019"
PC <- -1500 # critical leaf water potential for stomatal closure, J/kg
SP <- 10 # stability parameter for stomatal closure, -
# set microclima = 1 to get partitioned diffuse and direct
# solar
microclima <- 1
# - needs internet connection for the DEM download

```

Now run the simulation and save the results (or run next chunk if saved previously).

```

maxshade <- 90 # maximum shade available
micro_highshade <- micro_silo(loc = loc, dstart = dstart, dfinish = dfinish,
  Ushrht = Ushrht, microclima = microclima, maxshade = maxshade,
  PC = PC, SP = SP, email = "m. Kearney@unimelb.edu.au")
micro_highshade_orig <- micro_highshade # save a copy because will be modifying it later
maxshade <- 5 # maximum shade available
micro_lowshade <- micro_silo(loc = loc, dstart = dstart, dfinish = dfinish,
  Ushrht = Ushrht, microclima = 1, maxshade = maxshade, PC = PC,
  SP = SP, email = "m. Kearney@unimelb.edu.au")
micro_lowshade_orig <- micro_lowshade # save a copy because will be modifying it later
save(micro_highshade, file = "micro_highshade.Rda")
save(micro_lowshade, file = "micro_lowshade.Rda")

```

Optionally, can manipulate rainfall at a particular time to simulate artificial watering.

```

RAINFALL <- micro_highshade_orig$RAINFALL # extract rainfall from previous sim
dates2 <- micro_highshade_orig$dates2
dates <- micro_highshade_orig$dates
plot(dates2, RAINFALL, type = "h")
date_waters <- as.character(seq(as.POSIXct("2019-01-01"), as.POSIXct("2019-01-31"),
  by = "days"))
water <- 10 # mm water to add
RAINFALL2 <- RAINFALL
RAINFALL2[as.character(dates2) %in% date_waters] <- water
plot(dates2, RAINFALL2, type = "h")
# Create hourly values
rainhour <- unlist(lapply(RAINFALL2, function(x) c(x, rep(0,
  23)))))
maxshade <- 90 # maximum shade available
micro_highshade_water <- micro_silo(loc = loc, dstart = dstart,
  dfinish = dfinish, Ushrht = Ushrht, microclima = microclima,
  maxshade = maxshade, PC = PC, SP = SP, rainhourly = 1, rainhour = rainhour,
  email = "m. Kearney@unimelb.edu.au")
micro_highshade_orig <- micro_highshade_water # save a copy because will be modifying it later
maxshade <- 5 # maximum shade available
micro_lowshade_water <- micro_silo(loc = loc, dstart = dstart,
  dfinish = dfinish, Ushrht = Ushrht, microclima = 1, maxshade = maxshade,
  PC = PC, SP = SP, rainhourly = 1, rainhour = rainhour, email = "m. Kearney@unimelb.edu.au")
micro_lowshade_water_orig <- micro_lowshade_water # save a copy because will be modifying it later
save(micro_highshade_water, file = "micro_highshade_water.Rda")
save(micro_lowshade_water, file = "micro_lowshade_water.Rda")

soilmoist <- as.data.frame(micro_highshade$soilmoist)

```

```

soilmoist_water <- as.data.frame(micro_highshade_water$soilmoist)
plot(dates, soilmoist$WC20cm, type = "l") #plot 20 cm soil moisture
points(dates, soilmoist_water$WC20cm, type = "l", col = "blue")

load("micro_highshade.Rda")
load("micro_lowshade.Rda")
micro_lowshade_orig <- micro_lowshade # save a copy because will be modifying it later
micro_highshade_orig <- micro_highshade # save a copy because will be modifying it later
load("micro_highshade_water.Rda")
load("micro_lowshade_water.Rda")
micro_lowshade_water_orig <- micro_lowshade_water # save a copy because will be modifying it later
micro_highshade_water_orig <- micro_highshade_water # save a copy because will be modifying it later
dates2 <- micro_highshade_orig$dates2
dates <- micro_highshade_orig$dates

```

The results are returned as a list and are put in an object called ‘micro’. The ‘ectotherm’ function by default expects that ‘micro’ exists and passes items of that list to the microclimate model.

The aboveground microclimatic conditions are in a matrix called ‘metout’. The soil temperatures, including surface temperature, are in the ‘soil’ matrix. Note that the argument ‘microclima’ is set to 1, which means it uses code from the microclima package (Maclean et al., 2019) to partition solar radiation into direct and diffuse, and the diffuse fraction is then part of the microclimate model output. Pressure is obtained for the elevation used in the simulation.

```

micro <- micro_highshade # choose a microclimate object to use (ectotherm looks for 'micro')
metout <- as.data.frame(micro$metout) # microclimate aboveground conditions, 0% shade
shadmet <- as.data.frame(micro$shadmet) # microclimate aboveground conditions, 90% shade
soil <- as.data.frame(micro$soil) # microclimate soil conditions, 0% shade
shadsoil <- as.data.frame(micro$shadsoil) # microclimate soil conditions, 90% shade
PDIFs <- micro$diffuse_frac # use variable diffuse fraction
P_a <- get_pressure(micro$elev) # air pressure, Pa
P_a <- rep(P_a, nrow(metout)) # create hourly vector
dates <- micro$dates # dates

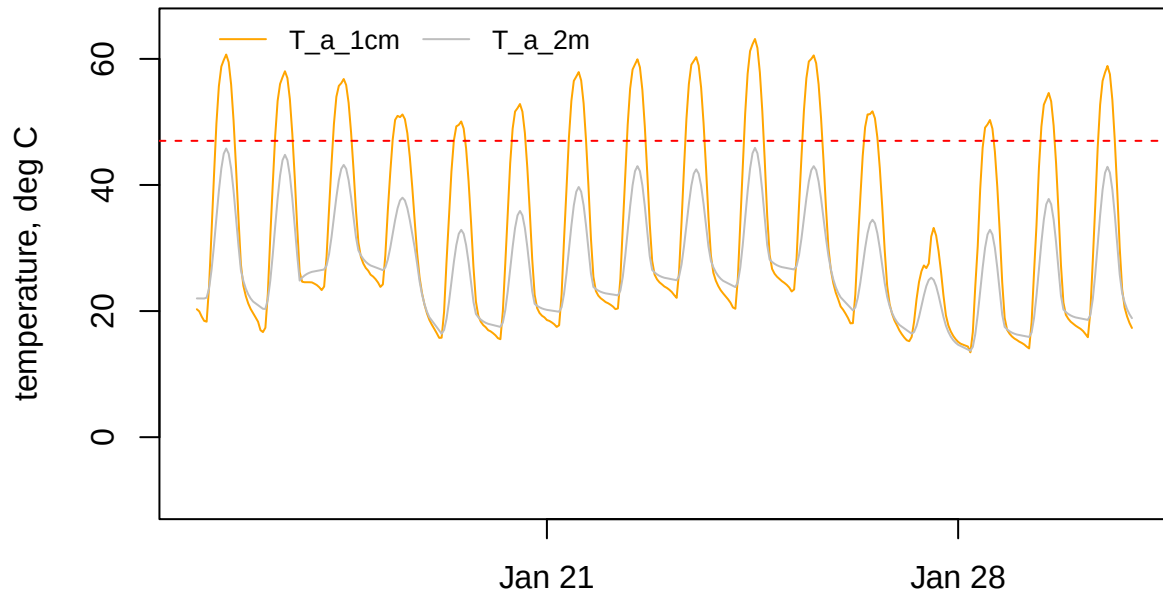
```

Plot air temperature during a heat wave.

```

# sub <- which(dates > as.POSIXct('2018-09-01') & dates <
# as.POSIXct('2019-03-31'))
sub <- which(dates > as.POSIXct("2019-01-15") & dates < as.POSIXct("2019-01-31"))
sub2 <- which(dates2 > as.POSIXct("2019-01-15") & dates < as.POSIXct("2019-01-31"))
plot(dates[sub], metout$TALOC[sub], type = "l", ylab = "temperature, deg C",
     xlab = "", col = "orange", ylim = c(-10, 65))
points(dates[sub], metout$TAREF[sub], type = "l", col = "grey")
abline(h = 47, col = "red", lty = 2)
legend(dates[sub][1], 68, legend = c("T_a_1cm", "T_a_2m"), col = c("orange",
"grey"), lty = 1, bty = "n", horiz = TRUE, cex = 0.8)

```



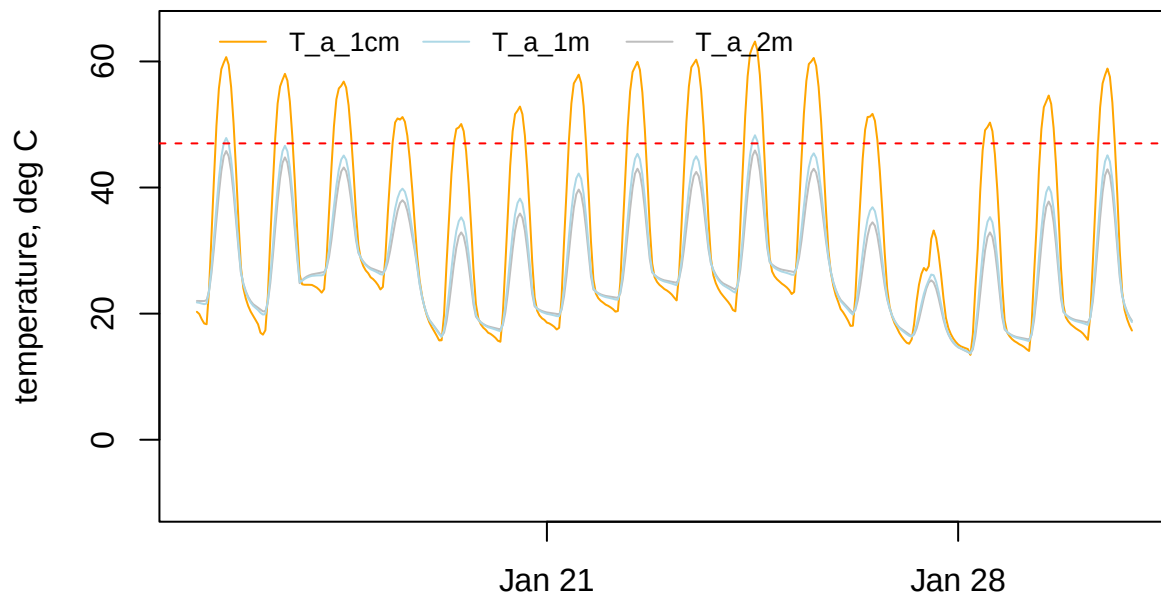
Example of getting air temperature, wind speed and humidity at a different height, choosing 1 m for a grasshopper on a small bush.

```
heights <- 1
new_TAs <- NULL
new_RHs <- NULL
new_VELs <- NULL
for (j in 1:nrow(metout)) {
  profile.out <- get_profile(Refhyt = 2, RUF = 0.004, heights = heights,
    TAREF = metout$TAREF[j], VREF = metout$VREF[j], RH = metout$RH[j],
    D0cm = soil$D0cm[j], ZEN = metout$ZEN[j], warn = FALSE)
  new_TAs[j] <- profile.out$TAs[2] # first output is at ground level
  new_RHs[j] <- profile.out$RHs[2] # first output is at ground level
  new_VELs[j] <- profile.out$VELs[2] # first output is at ground level
}
new_TAs_shade <- NULL
new_RHs_shade <- NULL
new_VELs_shade <- NULL
for (j in 1:nrow(metout)) {
  profile.out <- get_profile(Refhyt = 2, RUF = 0.004, heights = heights,
    TAREF = shadmet$TAREF[j], VREF = shadmet$VREF[j], RH = shadmet$RH[j],
    D0cm = shadsoil$D0cm[j], ZEN = shadmet$ZEN[j], warn = FALSE)
  new_TAs_shade[j] <- profile.out$TAs[2] # first output is at ground level
  new_RHs_shade[j] <- profile.out$RHs[2] # first output is at ground level
  new_VELs_shade[j] <- profile.out$VELs[2] # first output is at ground level
}
```

```

plot(dates[sub], metout$TALOC[sub], type = "l", ylab = "temperature, deg C",
     xlab = "", col = "orange", ylim = c(-10, 65))
points(dates[sub], metout$TAREF[sub], type = "l", col = "grey")
points(dates[sub], new_TAs[sub], type = "l", col = "lightblue")
abline(h = 47, col = "red", lty = 2)
legend(dates[sub][1], 68, legend = c("T_a_1cm", "T_a_1m", "T_a_2m"),
      col = c("orange", "lightblue", "grey"), lty = 1, bty = "n",
      horiz = TRUE, cex = 0.8)

```



Run ectotherm model for a grasshopper and a lizard

First the grasshopper, at 1 m height.

```

micro$metout[, 3] <- new_TAs
micro$metout[, 5] <- new_RHs
micro$metout[, 7] <- new_VELs
micro$shadmet[, 3] <- new_TAs_shade
micro$shadmet[, 5] <- new_RHs_shade
micro$shadmet[, 7] <- new_VELs_shade

```

```

Ww_g <- 0.2 # g, body mass
shape <- 1 # cylinder
shape_b <- 10 # ratio of long to short axis of cylinder
T_pref <- 38 # preferred body temperature (animal will attempt to regulate as close to this value as p

```

```

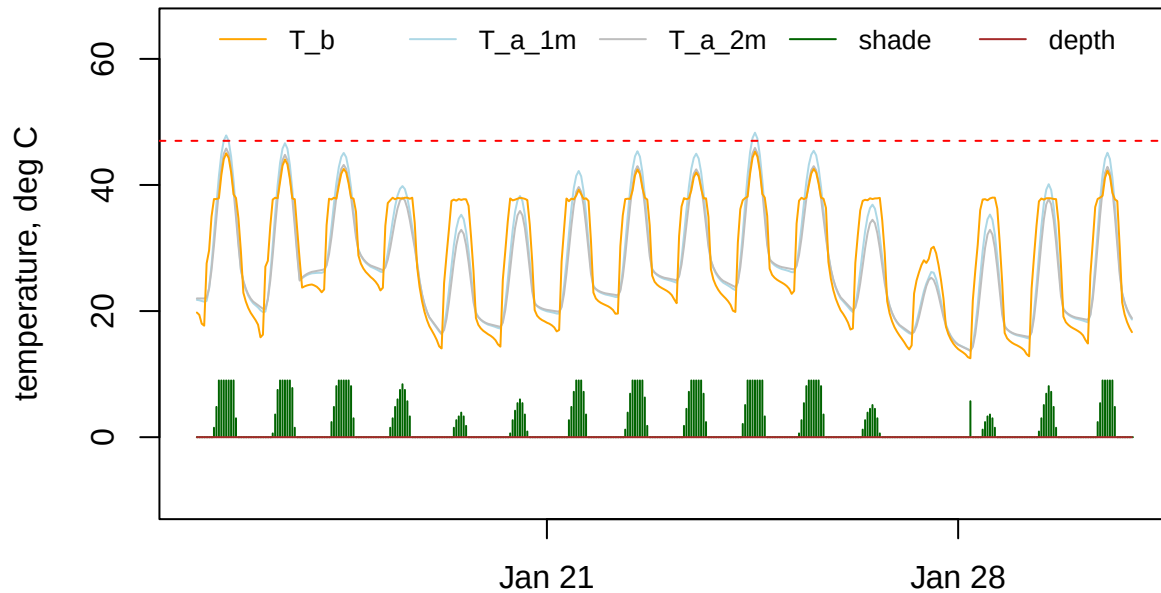
T_F_min <- 13 # degrees C, lower body temperature for foraging
T_F_max <- 40 # degrees C, upper body temperature for foraging
T_B_min <- 12.5 # degrees C, minimum basking temperature
T_RB_min <- 12.5 # degrees C, temperature at which animal will move to a basking site
CT_max <- 47 # degrees C, critical thermal maximum (animal will die if CT_kill = 1 and this threshold is reached)
CT_min <- 0 # degrees C, critical thermal minimum (used by program to determine depth selected when in water)

# behavioural traits (made stage-specific later)
diurn <- 1 # diurnal activity allowed (1) or not (0)?
nocturn <- 1 # nocturnal activity allowed (1) or not (0)?
crepus <- 1 # crepuscular activity allowed (1) or not (0)?
shade_seek <- 1 # shade seeking allowed (1) or not (0)?
burrow <- 0 # shelter in burrow allowed (1) or not (0)?
pct_cond <- 0 # Percentage of animal surface contacting the substrate (%)

ecto <- ectotherm(Ww_g = Ww_g, shape = shape, shape_b = shape_b,
  T_pref = T_pref, T_F_min = T_F_min, T_F_max = T_F_max, T_B_min = T_B_min,
  T_RB_min = T_RB_min, CT_min = CT_min, CT_max = CT_max, diurn = diurn,
  nocturn = nocturn, crepus = crepus, shade_seek = shade_seek,
  burrow = burrow, pct_cond = pct_cond)
environ <- as.data.frame(ecto$environ)

plot(dates[sub], new_TAs[sub], type = "l", ylab = "temperature, deg C",
  xlab = "", col = "lightblue", ylim = c(-10, 65))
points(dates[sub], metout$TAREF[sub], type = "l", col = "grey")
points(dates[sub], environ$TC[sub], type = "l", col = "orange")
points(dates[sub], environ$SHADE[sub]/10, type = "h", col = "darkgreen")
points(dates[sub], environ$DEP[sub]/10, type = "l", col = "brown")
abline(h = 47, col = "red", lty = 2)
legend(dates[sub][1], 68, legend = c("T_b", "T_a_1m", "T_a_2m",
  "shade", "depth"), col = c("orange", "lightblue", "grey",
  "darkgreen", "brown"), lty = 1, bty = "n", horiz = TRUE,
  cex = 0.8)

```



Now the lizard at 1 cm height. Resetting to original microclimate output.

```
micro <- micro_highshade_orig # original output at 1cm
Ww_g <- 40 # g, body mass
shape <- 4 # lizard
T_pref <- 41 # preferred body temperature (animal will attempt to regulate as close to this value as p
T_F_min <- 30 # degrees C, lower body temperature for foraging
T_F_max <- 44 # degrees C, upper body temperature for foraging
T_B_min <- 20 # degrees C, minimum basking temperature
T_RB_min <- 13 # degrees C, temperature at which animal will move to a basking site
CT_max <- 47 # degrees C, critical thermal maximum (animal will die if CT_kill = 1 and this threshold
CT_min <- 0 # degrees C, critical thermal minimum (used by program to determine depth selected when in

# behavioural traits (made stage-specific later)
diurn <- 1 # diurnal activity allowed (1) or not (0)?
nocturn <- 0 # nocturnal activity allowed (1) or not (0)?
crepus <- 0 # crepuscular activity allowed (1) or not (0)?
shade_seek <- 1 # shade seeking allowed (1) or not (0)?
burrow <- 1 # shelter in burrow allowed (1) or not (0)?
pct_cond <- 10 # Percentage of animal surface contacting the substrate (%)

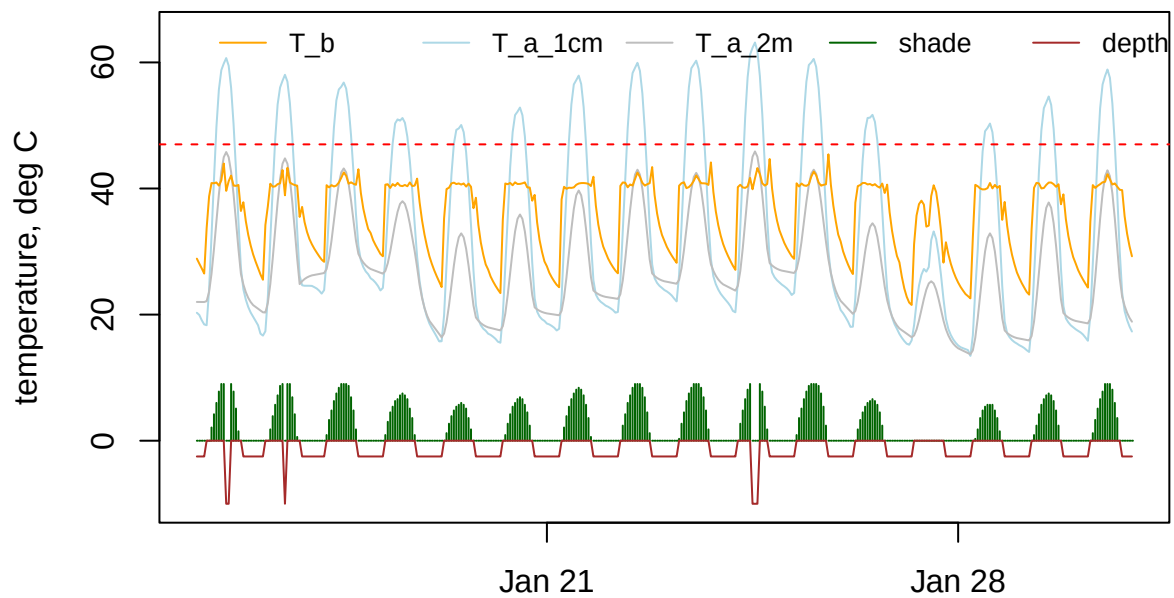
ecto <- ectotherm(Ww_g = Ww_g, shape = shape, shape_b = shape_b,
  T_pref = T_pref, T_F_min = T_F_min, T_F_max = T_F_max, T_B_min = T_B_min,
  T_RB_min = T_RB_min, CT_min = CT_min, CT_max = CT_max, diurn = diurn,
  nocturn = nocturn, crepus = crepus, shade_seek = shade_seek,
```

```

burrow = burrow, pct_cond = pct_cond)
environ <- as.data.frame(ecto$environ)

plot(dates[sub], metout$TALOC[sub], type = "l", ylab = "temperature, deg C",
     xlab = "", col = "lightblue", ylim = c(-10, 65))
points(dates[sub], metout$TAREF[sub], type = "l", col = "grey")
points(dates[sub], environ$TC[sub], type = "l", col = "orange")
points(dates[sub], environ$SHADE[sub]/10, type = "h", col = "darkgreen")
points(dates[sub], environ$DEP[sub], type = "l", col = "brown")
abline(h = 47, col = "red", lty = 2)
legend(dates[sub][1], 68, legend = c("T_b", "T_a_1cm", "T_a_2m",
  "shade", "depth"), col = c("orange", "lightblue", "grey",
  "darkgreen", "brown"), lty = 1, bty = "n", horiz = TRUE,
  cex = 0.8)

```



Define ectotherm model settings for leaf temperature

To simulate a leaf we need to turn off the behaviour and metabolism used for simulating animals. This can be achieved most easily by setting 'live' to 0. Setting 'leaf' to 1 means the model uses stomatal conductance to compute evaporation.

```

# model settings
live <- 0 # don't simulate behaviour or respiration
leaf <- 1 # use the stomatal conductance formulation for evaporation

```


Also go back to 1 m height.

```
micro$metout[, 3] <- new_TAs
micro$metout[, 5] <- new_RHs
micro$metout[, 7] <- new_VELs
micro$shadmet[, 3] <- new_TAs_shade
micro$shadmet[, 5] <- new_RHs_shade
micro$shadmet[, 7] <- new_VELs_shade
```

Define leaf functional traits

Next the traits of the leaf must be defined. An ellipsoid shape is used in this simulation. Specification of the mass and the ratios of the linear dimensions of the shape (here, semi-minor axes relative to the semi-major axis) together with the density (default value 1 g cm³) allows the geometric functions in the ectotherm model to compute the absolute dimensions and surface areas.

Evaporative exchange is driven by the stomatal conductance parameters, which can be specified for the abaxial and adaxial sides of the leaf as well as the overall base leaf conductance through the cuticle. The 'pct_wet' parameter is set to 0.

Radiative exchange parameters can be set including the emissivities of the leaf and the substrate, the solar absorptivities of the leaf, the configuration factors to the sky and ground, the percentage of the leaf conducting to the ground and the orientation to the sun.

```
# leaf functional traits
Ww_g <- 1 # wet weight, g
shape <- 2 # 0=plate, 1=cylinder, 2=ellipsoid
shape_b <- 0.0025 # ratio of b axis:a axis for ellipsoid
shape_c <- 0.1176 # ratio of c axis:a axis for ellipsoid
g_vs_ab <- 0.4 # leaf vapour conductance, abaxial (bottom of leaf), mol/m2/s
g_vs_ad <- 0 # leaf vapour conductance, adaxial (top of leaf), mol/m2/s
g_vs_ab_max <- 0.4 # leaf vapour conductance, abaxial (bottom of leaf), mol/m2/s
g_vs_ad_max <- 0 # leaf vapour conductance, adaxial (top of leaf), mol/m2/s
epsilon_sub <- 0.95 # emissivity of the substrate (-)
epsilon_sky <- 1 # emissivity of the sky (-), accounted for already in
# 'TSKY' output from microclimate model so make it 1
epsilon_L <- 0.97 # emissivity of the leaf (-)
alpha_L <- 0.5 # solar absorptivity of the leaf (-)
fatosk <- 0.5 # radiation configuration factor to sky (-)
fatosb <- 0.5 # radiation configuration factor to substrate (-)
conv_enhance <- 1.4 # convective enhancement factor (-)
pct_cond <- 0 # percent of leaf conducting to the ground (%)
postur <- 3 # leaf oriented horizontally, not tracking sun
M_1 <- 0 # make metabolic rate zero
```

The ectotherm model needs a vector of hourly stomatal conductance values. I.e., there is no 'on-the-fly' response of the stomata other than the stomata opening up to the maximum to avoid leaf temperature reaching the value specified by the argument 'CT_max' (default is 40 deg C if the model is run with 'leaf = 2'). Here we simply close the stomata at night, based on the zenith angle of the sun as calculated by the microclimate model.

```
# set up vapour conductance vectors and simulate stomatal
# closure at night
f_open <- rep(1, nrow(metout))
```

```
f_open[metout$ZEN == 90] <- 0 # close stomata when the sun is set
g_vs_abs <- g_vs_ab * f_open
g_vs_ads <- g_vs_ad * f_open
```

Run the NicheMapR ectotherm model

Now we can run the model for each hour of the middle day of each month (specified in the microclimate which, as mentioned above, is by default coming from the object 'micro' which is assumed to be present in the R environment).

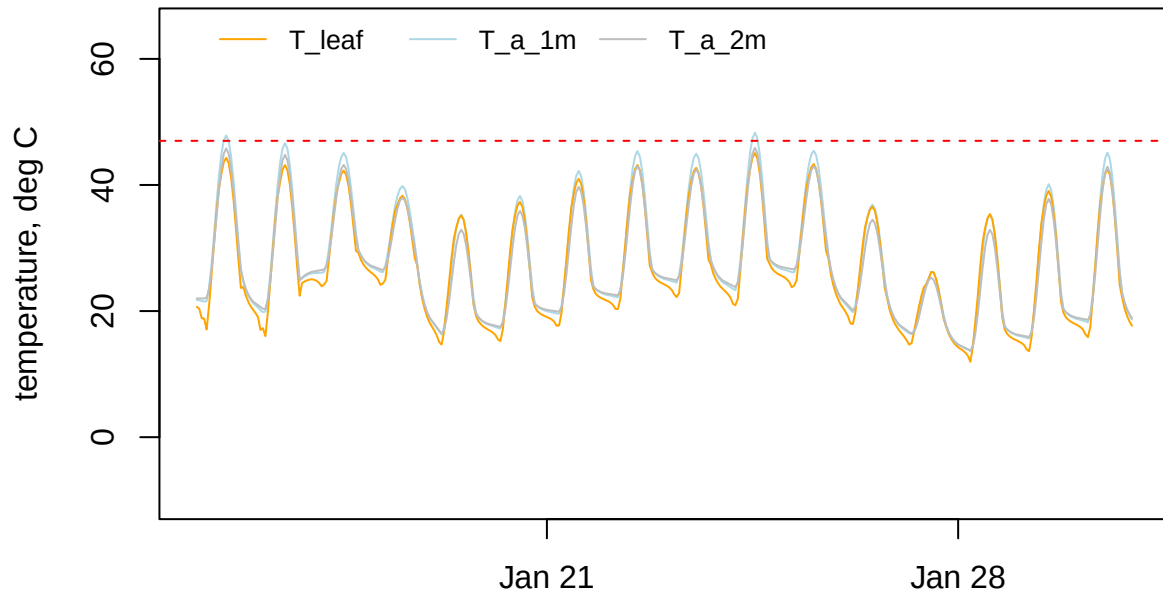
```
ptm <- proc.time() # Start timing
ecto <- ectotherm(leaf = leaf, live = live, Ww_g = Ww_g, shape = shape,
  shape_b = shape_b, shape_c = shape_c, g_vs_ab = g_vs_abs,
  g_vs_ad = g_vs_ads, g_vs_ab_max = g_vs_ab_max, g_vs_ad_max = g_vs_ad_max,
  epsilon_sub = epsilon_sub, epsilon_sky = epsilon_sky, epsilon = epsilon_L,
  alpha_max = alpha_L, alpha_min = alpha_L, M_1 = M_1, fatosk = fatosk,
  fatosb = fatosb, conv_enhance = conv_enhance, pct_cond = pct_cond,
  postur = postur, preshr = P_a, PDIF = PDIFs)
print(proc.time() - ptm) # Stop the clock
```

```
##      user  system elapsed
##    1.99    0.02    2.02
```

```
environ <- as.data.frame(ecto$environ)
T_leaf <- environ$TC
```

Now plot the results against air temperature at leaf height and at the reference height of 2 m

```
plot(dates[sub], new_TAs[sub], type = "l", ylab = "temperature, deg C",
  xlab = "", col = "lightblue", ylim = c(-10, 65))
points(dates[sub], T_leaf[sub], type = "l", col = "orange")
points(dates[sub], metout$TAREF[sub], type = "l", col = "grey")
abline(h = 47, col = "red", lty = 2)
legend(dates[sub][1], 68, legend = c("T_leaf", "T_a_1m", "T_a_2m"),
  col = c("orange", "lightblue", "grey"), lty = 1, bty = "n",
  horiz = TRUE, cex = 0.8)
```



Now integrate the plantecophys package to get more realistic stomatal behaviour via the ‘Photosyn’ function. This code runs Photosyn assuming the leaf temperature calculated with the ectotherm model to start with.

```
# now include simulated stomatal conductance from plantecophys
UMOLPERJ <- 4.57 # mu_mol photons / J
WETAIR_out <- WETAIR(db = new_TAs, rh = new_RHs, bp = P_a)
VPD_Pa <- WETAIR_out$esat - WETAIR_out$e
photo_out <- Photosyn(Tleaf = T_leaf,
  g1 = 5.2, # Parameter of Ball-Berry model
  Ca = 400, # Atmospheric CO2 concentration (ppm)
  VPD = VPD_Pa/1000, # Vapour pressure deficit (kPa)
  vpdmin = 0.5, # lower threshold on VPD
  PPFD = metout$SOLR * UMOLPERJ * 1, # mu mol m-2 s-1
  Patm = P_a/1000 # Atmospheric pressure (kPa)
)
g_vs_abs_pho <- photo_out$GS
```

Now recalculate leaf temperature using these stomatal conductance values from Photosyn.

```
ptm <- proc.time() # Start timing
ecto <- ectotherm(leaf = leaf, live = live, Ww_g = Ww_g, shape = shape,
  shape_b = shape_b, shape_c = shape_c, g_vs_ab = g_vs_abs_pho,
  g_vs_ad = g_vs_ads, g_vs_ab_max = g_vs_ab_max, g_vs_ad_max = g_vs_ad_max,
  epsilon_sub = epsilon_sub, epsilon_sky = epsilon_sky, epsilon = epsilon_L,
  alpha_max = alpha_L, alpha_min = alpha_L, M_1 = M_1, fatosk = fatosk,
  fatosb = fatosb, conv_enhance = conv_enhance, pct_cond = pct_cond,
```

```

    postur = postur, preshr = P_a, PDIF = PDIFs)
print(proc.time() - ptm) # Stop the clock

```

```

##      user  system elapsed
##    1.45    0.05     1.50

```

```

environ <- as.data.frame(ecto$environ)
T_leaf_plantecophys <- environ$TC

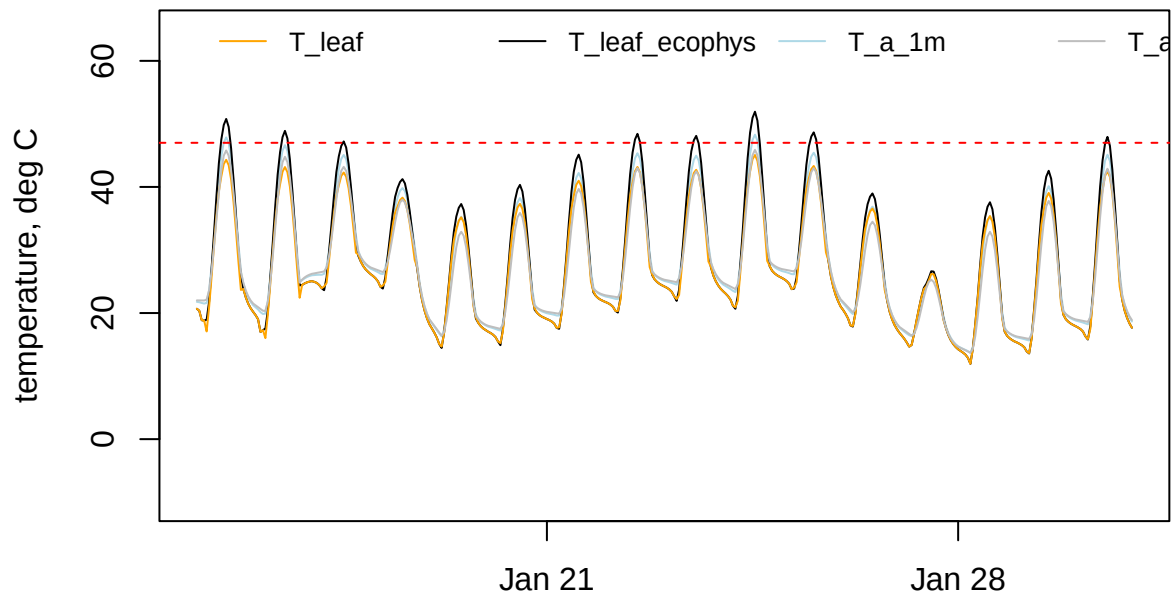
```

Compare the two calculations of leaf temperature. Stomatal closure in the middle of the day has caused leaf temperature to rise substantially.

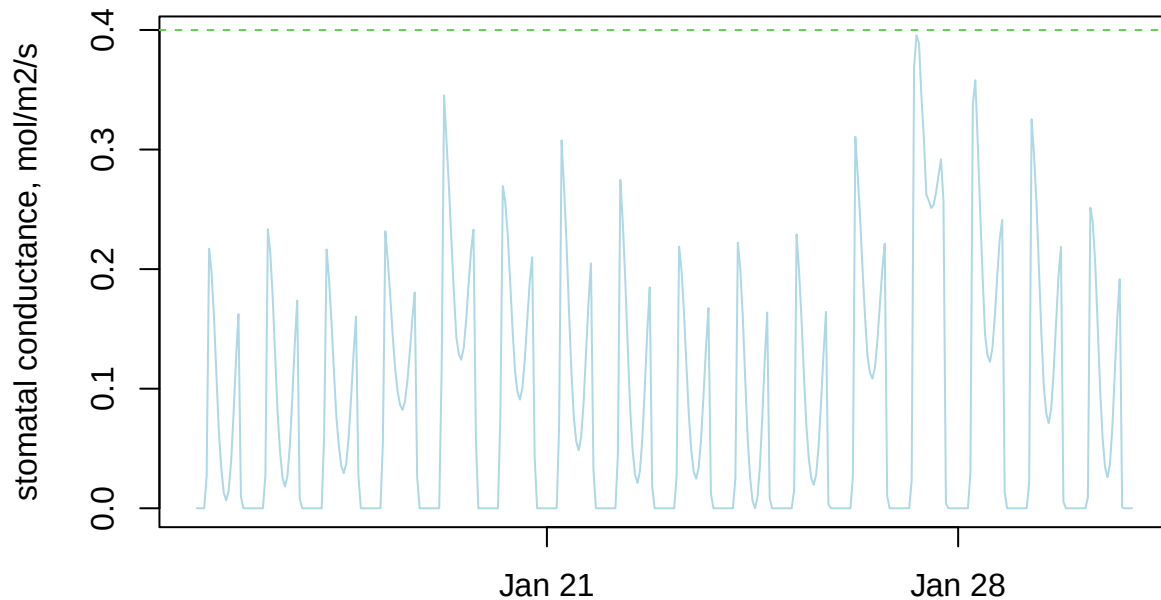
```

plot(dates[sub], new_TAs[sub], type = "l", ylab = "temperature, deg C",
     xlab = "", col = "lightblue", ylim = c(-10, 65))
points(dates[sub], T_leaf_plantecophys[sub], type = "l", col = "black",
       ylab = "temperature, deg C", ylim = c(-10, 65))
points(dates[sub], T_leaf[sub], type = "l", col = "orange", lty = 1)
points(dates[sub], metout$TAREF[sub], type = "l", col = "grey")
abline(h = 47, col = "red", lty = 2)
legend(dates[sub][1], 68, legend = c("T_leaf", "T_leaf_ecophys",
  "T_a_1m", "T_a_2m"), col = c("orange", "black", "lightblue",
  "grey"), lty = 1, bty = "n", horiz = TRUE, cex = 0.8)

```



```
plot(dates[sub], g_vs_abs_pho[sub], type = "l", ylab = "stomatal conductance, mol/m2/s",
     xlab = "", col = "lightblue")
abline(h = g_vs_ab_max, lty = 2, col = 3)
```



But, the problem is, we need to know the leaf temperature to determine stomatal behaviour and vice versa. So now we do a loop feeding the results back and forth between the two functions until we get convergence.

```
# now do a loop of n iterations to see when convergence
# occurs

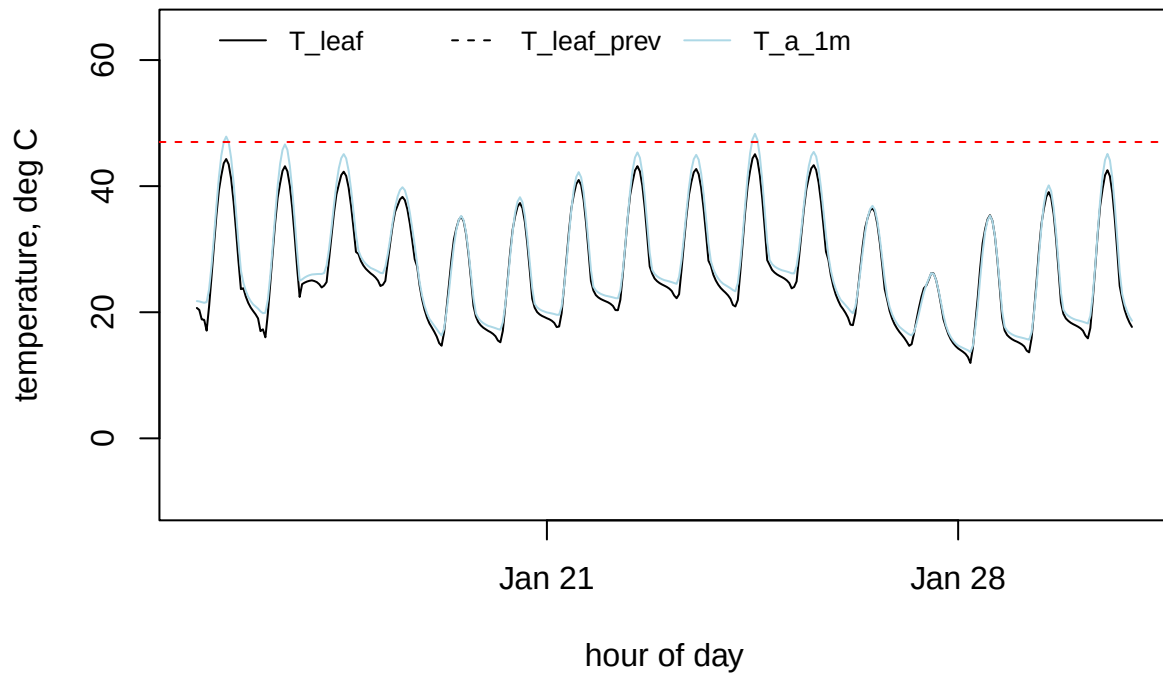
n <- 6 # iterations to do to find steady state stomatal conductance
# par(mfrow = c(2, 3))
g_vs_abs_iter <- g_vs_abs
T_leaf_iter <- NULL
for (i in 1:n) {
  ecto <- ectotherm(leaf = leaf, live = live, Ww_g = Ww_g,
    shape = shape, shape_b = shape_b, shape_c = shape_c,
    g_vs_ab = g_vs_abs_iter, g_vs_ad = g_vs_ads, g_vs_ab_max = g_vs_ab_max,
    g_vs_ad_max = g_vs_ad_max, epsilon_sub = epsilon_sub,
    epsilon_sky = epsilon_sky, epsilon = epsilon_L, alpha_max = alpha_L,
    alpha_min = alpha_L, M_1 = M_1, fatosk = fatosk, fatosb = fatosb,
    conv_enhance = conv_enhance, pct_cond = pct_cond, postur = postur,
    preshr = P_a, PDIF = PDIFs)
  environ <- as.data.frame(ecto$environ)
  if (i > 1) {
    T_leaf_prev <- T_leaf_iter
```

```

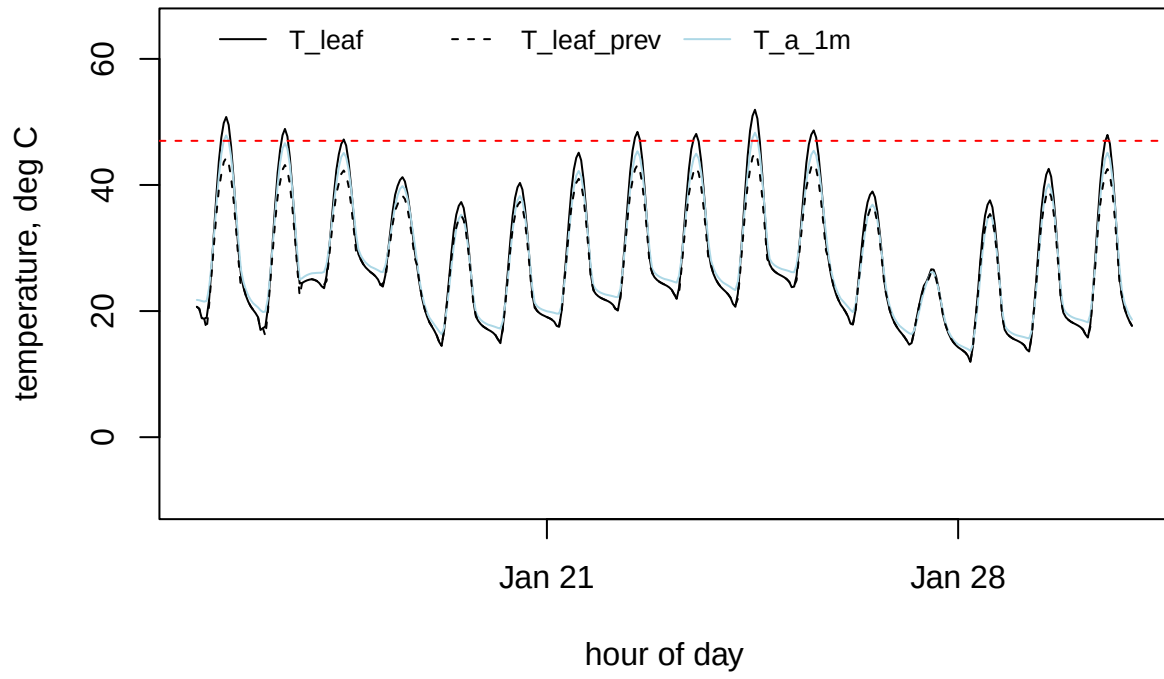
}
T_leaf_iter <- environ$TC
photo_out <- Photosyn(Tleaf = T_leaf_iter, g1 = 5.2, Ca = 400,
  VPD = VPD_Pa/1000, vpdmin = 0.5, PPFD = metout$SOLR *
  UMOLPERJ * 1, Patm = P_a/1000)
g_vs_abs_iter <- photo_out$GS
plot(dates[sub], T_leaf_iter[sub], type = "l", col = 1, ylim = c(-10,
  65), main = i, ylab = "temperature, deg C", xlab = "hour of day")
points(dates[sub], new_TAs[sub], type = "l", col = "lightblue")
if (i > 1) {
  points(dates[sub], T_leaf_prev[sub], type = "l", lty = 2)
}
abline(h = 47, col = "red", lty = 2)
legend(dates[sub][1], 68, legend = c("T_leaf", "T_leaf_prev",
  "T_a_1m"), col = c("black", "black", "lightblue"), lty = c(1,
  2, 1), bty = "n", horiz = TRUE, cex = 0.8)
}

```

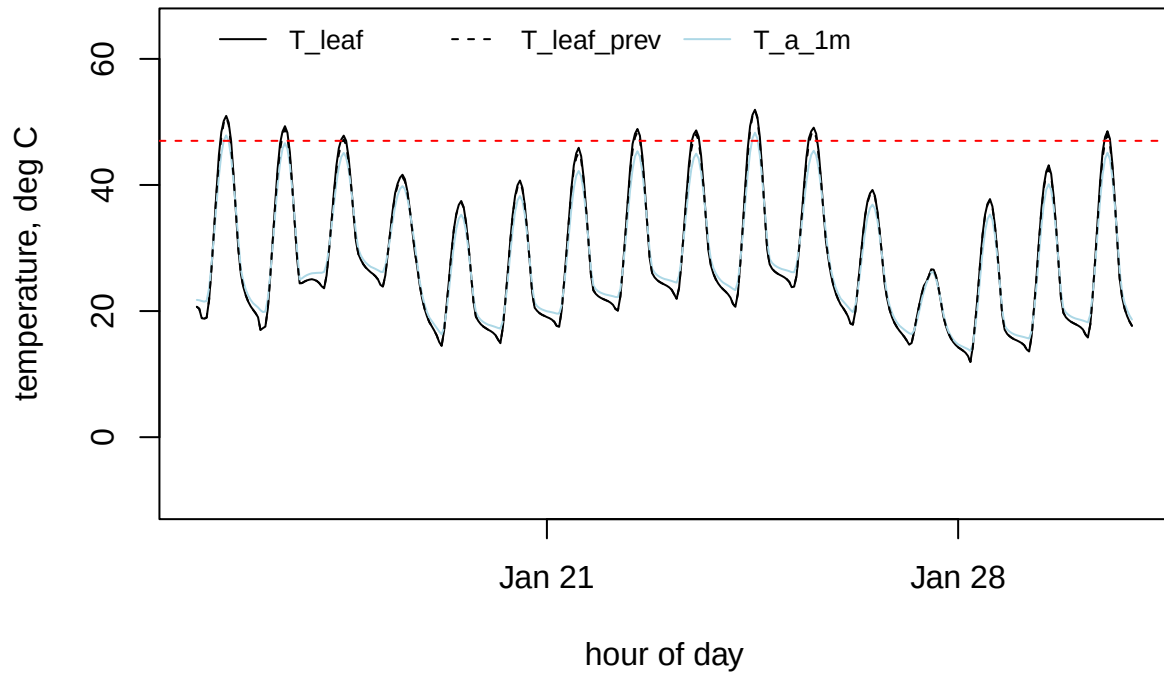
1



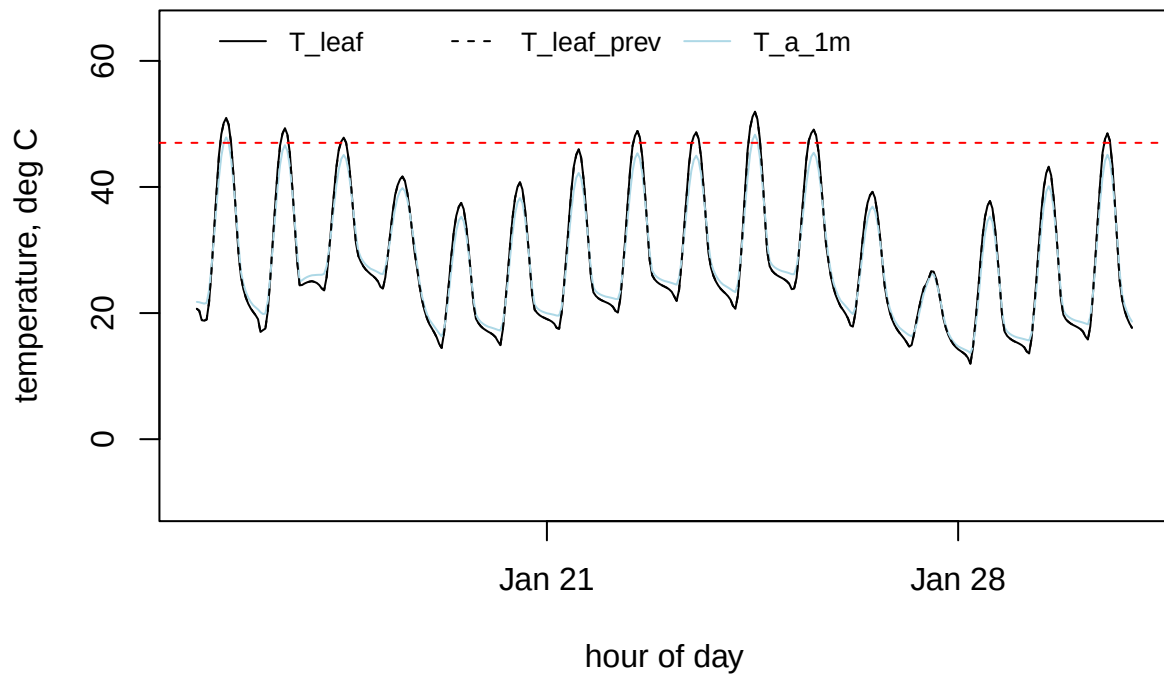
2



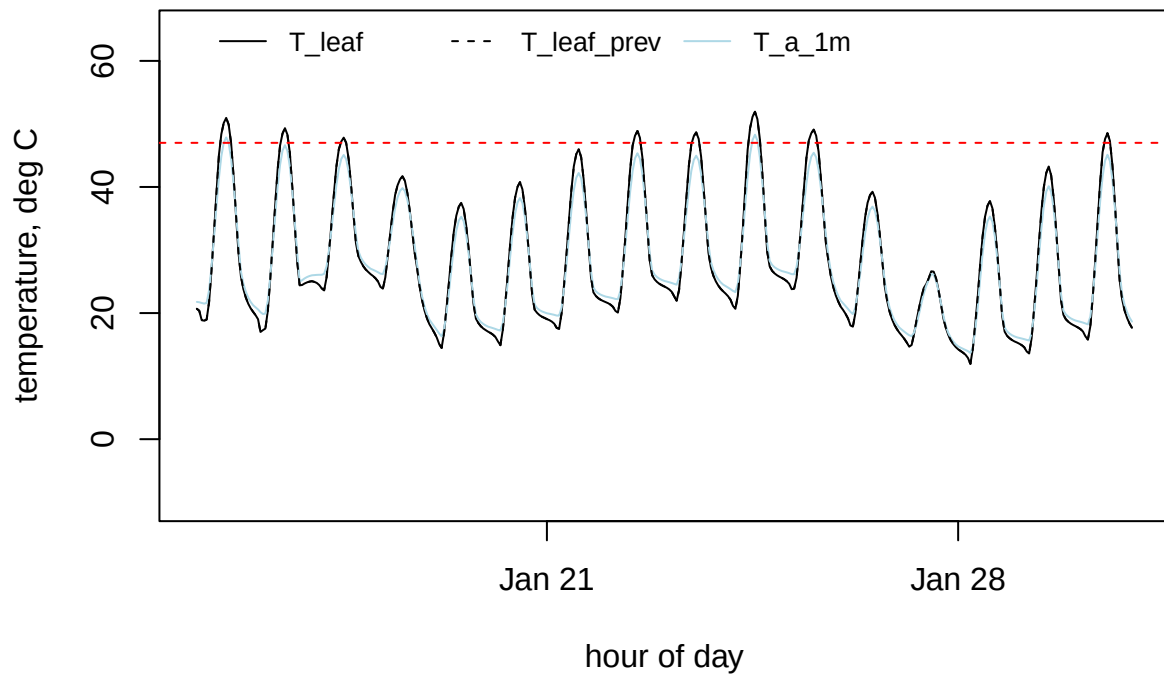
3

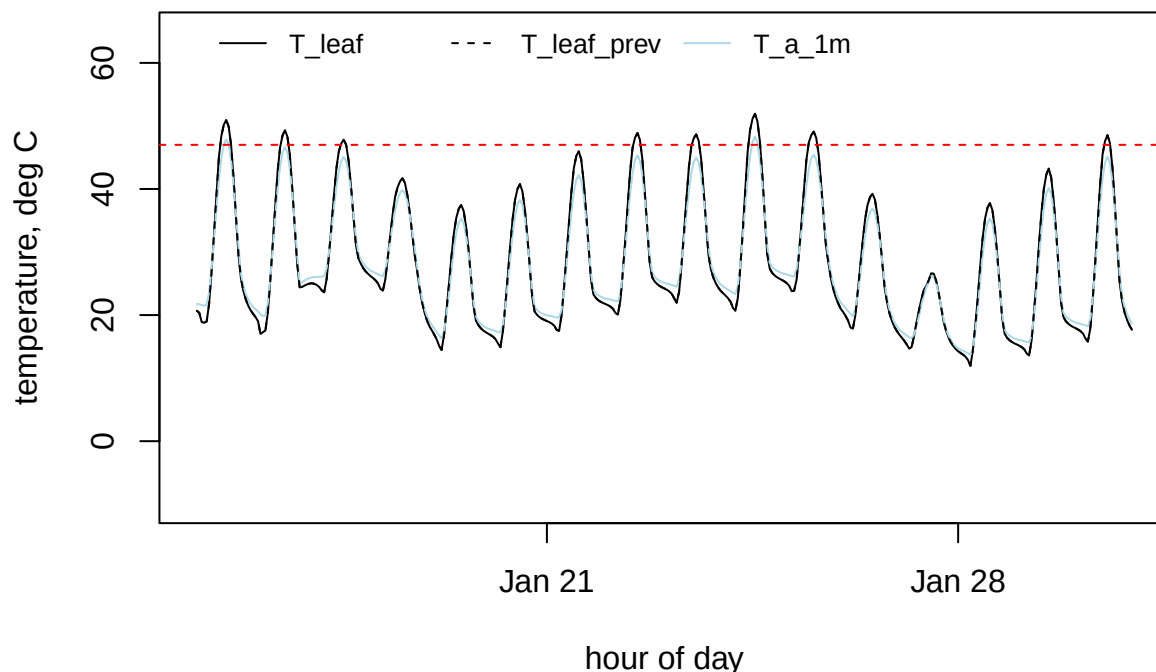


4



5





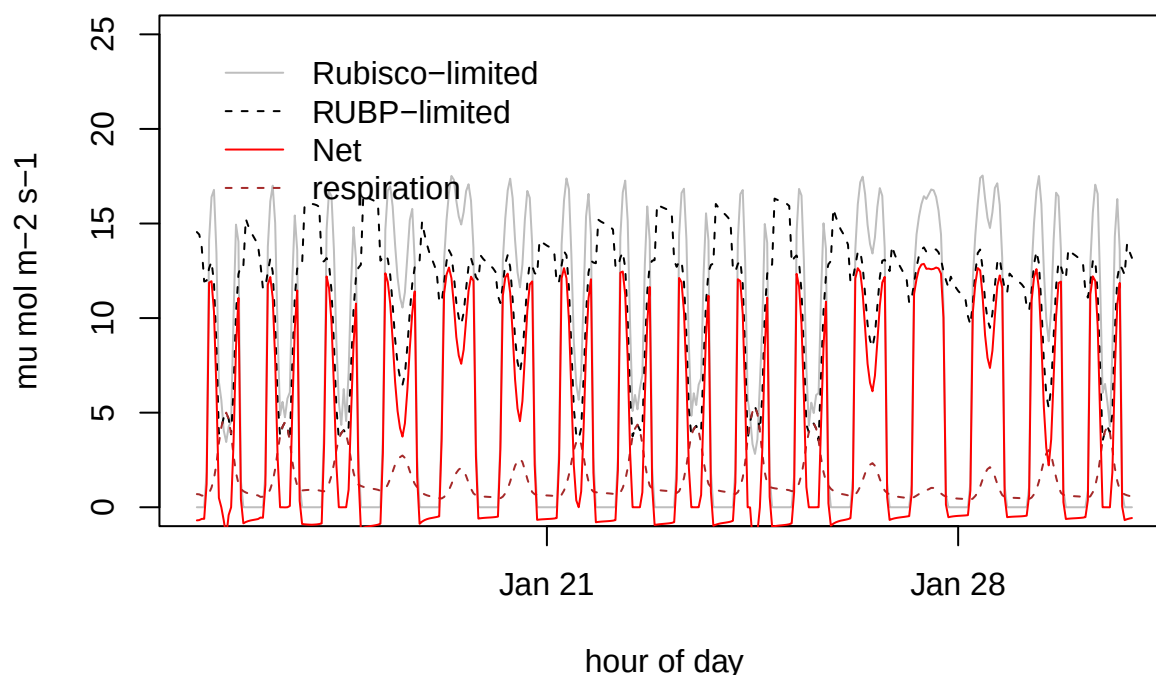
```
g_vs_abs_pho <- g_vs_abs_iter
T_leaf_pho <- environ$TC # final leaf temperature estimate
```

Photosynthesis consequences

We can look at the consequences for photosynthesis as predicted by the ‘Photosyn’ function. First we calculate and plot different aspects of metabolism with our finalised leaf temperature; the rubisco-limited photosynthesis, the RUBP-limited photosynthesis, the respiration and the net photosynthesis.

```
# plot photosynthesis outputs
par(mfrow = c(1, 1))
A_j <- photo_out$Aj
A_c <- photo_out$Ac
R_d <- photo_out$Rd
A_n <- photo_out$ALEAF # A_n = pmin(A_c, A_j) - R_d
plot(dates[sub], A_j[sub], type = "l", ylab = "mu mol m-2 s-1",
     main = "computed leaf temperature", xlab = "hour of day",
     ylim = c(0, 25), col = "grey")
points(dates[sub], A_c[sub], type = "l", lty = 2)
points(dates[sub], R_d[sub], type = "l", ylab = "mu mol m-2 s-1",
       col = "brown", lty = 2)
points(dates[sub], A_n[sub], type = "l", col = "red")
legend(dates[sub][1], 25, c("Rubisco-limited", "RUBP-limited",
                          "Net", "respiration"), col = c("grey", "black", "red", "brown"),
       lty = c(1, 2, 1, 2), bty = "n")
```

computed leaf temperature

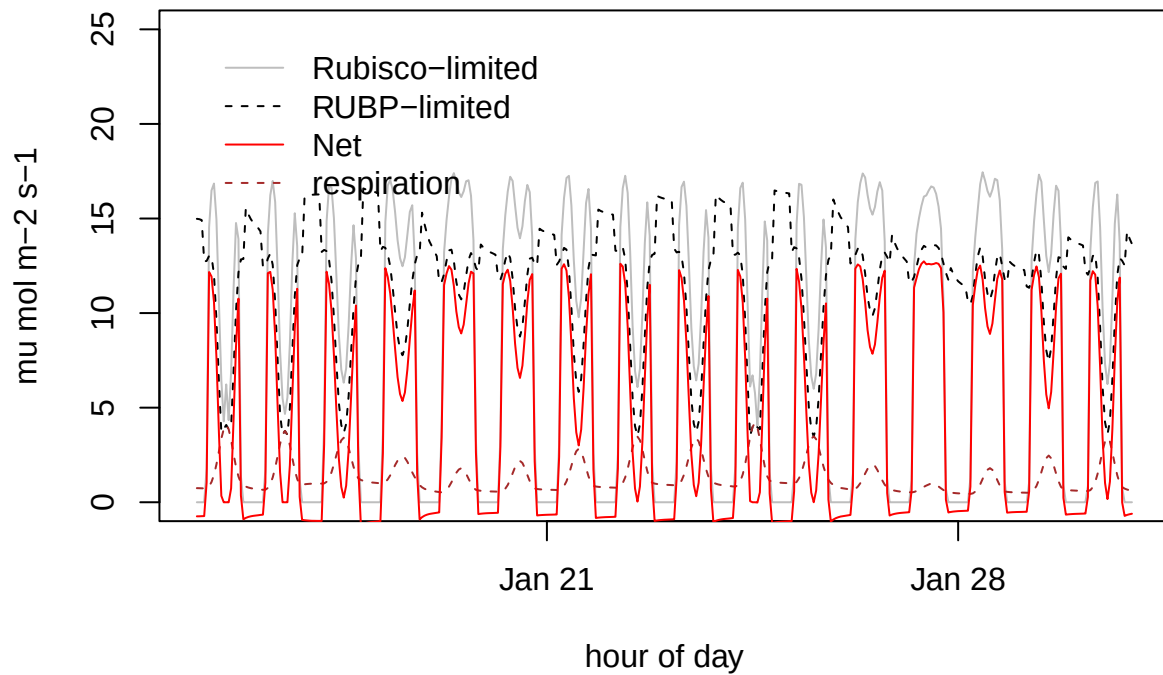


Now we can see what we'd have predicted if we assumed leaf temperature was equal to air temperature.

```
# redo with T_leaf = T_air
photo_out_T_air <- Photosyn(Tleaf = new_TAs, g1 = 5.2, Ca = 400,
  VPD = VPD_Pa/1000, vpdmin = 0.5, PPFD = metout$SOLR * UMOLPERJ *
  1, Patm = P_a/1000)

# plot photosynthesis outputs
A_j2 <- photo_out_T_air$Aj
A_c2 <- photo_out_T_air$Ac
R_d2 <- photo_out_T_air$Rd
A_n2 <- photo_out_T_air$ALEAF
plot(dates[sub], A_j2[sub], type = "l", ylab = "mu mol m-2 s-1",
  main = "leaf temperature = air temperature", xlab = "hour of day",
  ylim = c(0, 25), col = "grey")
points(dates[sub], A_c2[sub], type = "l", lty = 2)
points(dates[sub], R_d2[sub], type = "l", ylab = "mu mol m-2 s-1",
  col = "brown", lty = 2)
points(dates[sub], A_n2[sub], type = "l", col = "red")
legend(dates[sub][1], 25, c("Rubisco-limited", "RUBP-limited",
  "Net", "respiration"), col = c("grey", "black", "red", "brown"),
  lty = c(1, 2, 1, 2), bty = "n")
```

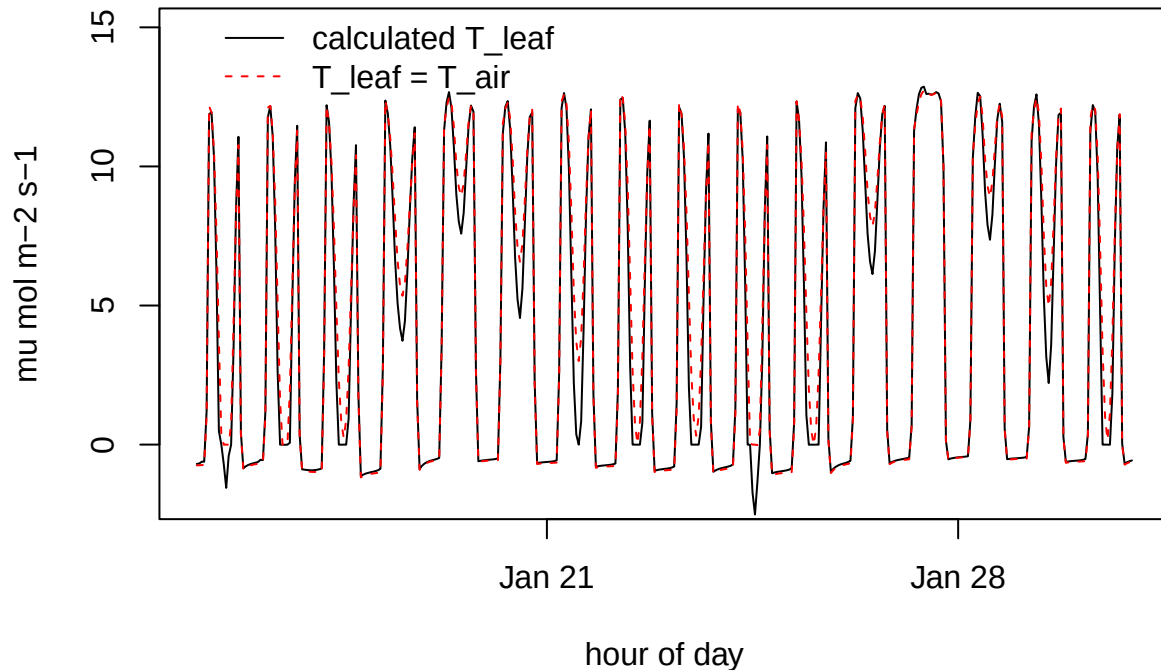
leaf temperature = air temperature



Finally, we can plot the difference in net photosynthesis.

```
# compare net photosynthesis using calculated leaf
# temperature or air temperature
plot(dates[sub], A_n[sub], type = "l", ylab = "mu mol m-2 s-1",
     main = "leaf vs air temperature", xlab = "hour of day", ylim = c(-2,
15))
points(dates[sub], A_n2[sub], type = "l", col = "red", lty = 2)
legend(dates[sub][1], 16, c("calculated T_leaf", "T_leaf = T_air"),
     col = c("black", "red"), lty = c(1, 2), bty = "n")
```

leaf vs air temperature



Soil moisture stress and stomatal closure

Next we use the SPAC model embedded in the microclimate model to assess when stomata would be simulated to close due to reaching a critical leaf water potential (due to the risk of cavitation, i.e. bubbles in the xylem).

The SPAC model is in the microclimate model to properly account for transpiration when simulating soil moisture. It includes the parameters ‘PC’, the critical leaf water potential for stomatal closure, and ‘SP’, the stability parameter for stomatal closure. The idea is to drive our simulated leaf’s stomatal closure by what the SPAC model did in response to the calculated soil moisture during the microclimate simulation.

Extract the plant predictions (from the SPAC model) from the ‘micro’ object.

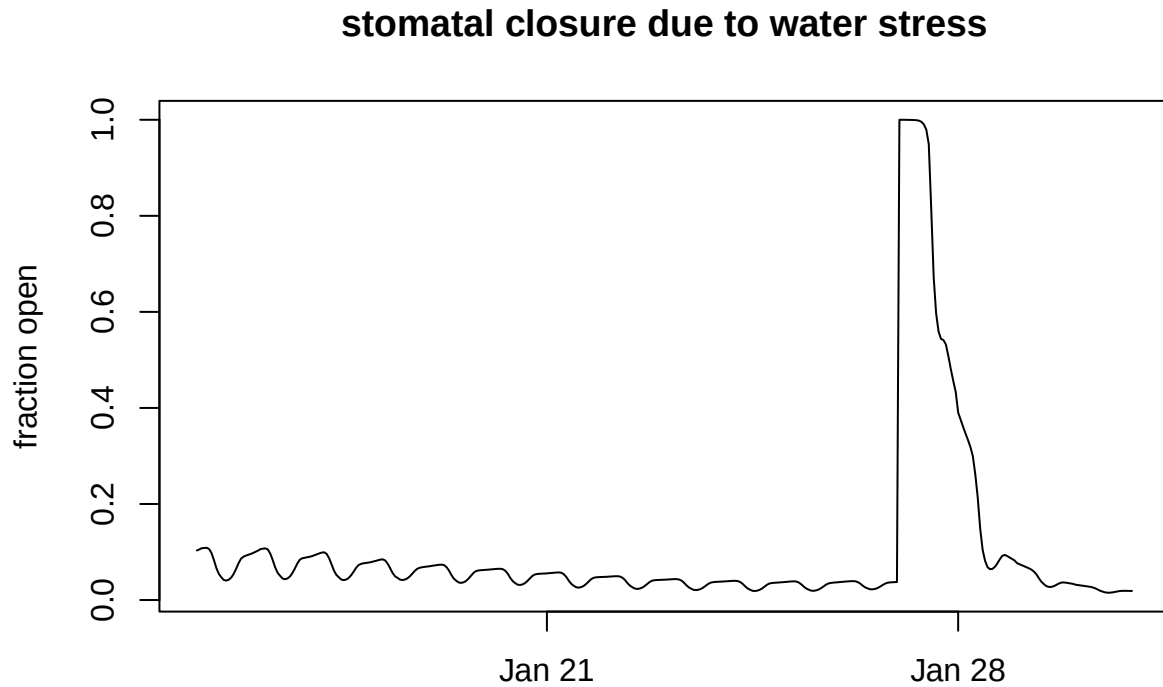
```
plant <- as.data.frame(micro$plant)
shadplant <- as.data.frame(micro$shadplant)
```

Compute stomatal dynamics with Photosyn

The ‘plant’ object contains the hourly predictions of transpiration rate, leaf water potential and root water potentials (for each depth in the soil). We need the leaf water potential to reconstruct the scaling factor (0 to 1) used in the SPAC model (see eq. 15 in Appendix 1). We will use this scaling factor as a metric of stomatal openness.

```
# calculate stomata openness as a function of leaf water
# potential
f_open <- 1/(1 + (plant$LPT/PC)^SP)
```

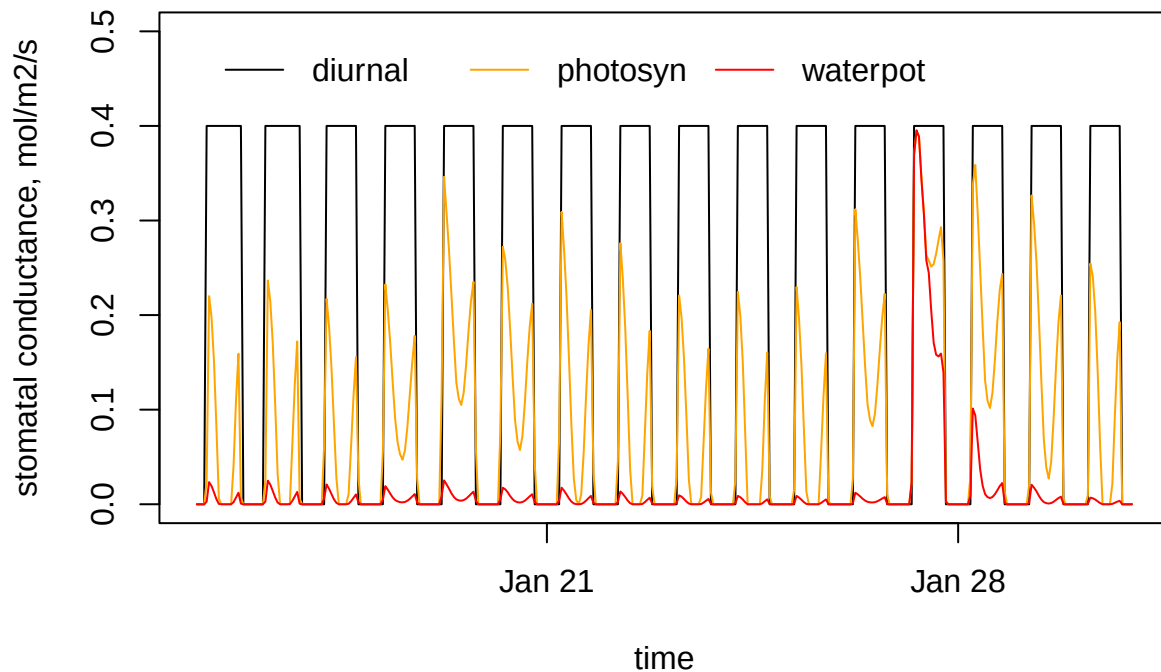
```
plot(dates[sub], f_open[sub], type = "l", ylab = "fraction open",
     xlab = "", main = "stomatal closure due to water stress")
```



Now we can adjust stomatal conductances so that the photosynthesis dynamics are overridden by the dynamics in response to leaf water stress.

```
# impose water stress effects
g_vs_abs_pho_pot <- g_vs_abs_pho
g_vs_abs_pho_pot[g_vs_abs_pho * f_open < g_vs_abs_pho] <- g_vs_abs_pho[g_vs_abs_pho *
  f_open < g_vs_abs_pho] * f_open[g_vs_abs_pho * f_open < g_vs_abs_pho]

plot(dates[sub], g_vs_abs[sub], type = "l", ylab = "stomatal conductance, mol/m2/s",
     xlab = "time", ylim = c(0, 0.5))
points(dates[sub], g_vs_abs_pho[sub], type = "l", col = "orange")
points(dates[sub], g_vs_abs_pho_pot[sub], type = "l", col = "red")
legend(dates[sub][1], 0.5, c("diurnal", "photosyn", "waterpot"),
      col = c("black", "orange", "red"), lty = 1, hori = TRUE,
      bty = "n")
```



Now we can run the ectotherm model with these adjusted stomatal dynamics, iterating to get convergence.

```
n <- 3 # iterations to do to find steady state stomatal conductance
# par(mfrow = c(2, 3))
g_vs_abs_iter <- g_vs_abs_pho_pot
T_leaf_iter <- NULL
for (i in 1:n) {
  ecto <- ectotherm(leaf = leaf, live = live, Ww_g = Ww_g,
    shape = shape, shape_b = shape_b, shape_c = shape_c,
    g_vs_ab = g_vs_abs_iter, g_vs_ad = g_vs_ads, g_vs_ab_max = g_vs_ab_max,
    g_vs_ad_max = g_vs_ad_max, epsilon_sub = epsilon_sub,
    epsilon_sky = epsilon_sky, epsilon = epsilon_L, alpha_max = alpha_L,
    alpha_min = alpha_L, M_1 = M_1, fatosk = fatosk, fatosb = fatosb,
    conv_enhance = conv_enhance, pct_cond = pct_cond, postur = postur,
    preshr = P_a, PDIF = PDIFs)
  environ <- as.data.frame(ecto$environ)
  if (i > 1) {
    T_leaf_prev <- T_leaf_iter
  }
  T_leaf_iter <- environ$TC
  photo_out <- Photosyn(Tleaf = T_leaf_iter, g1 = 5.2, Ca = 400,
    VPD = VPD_Pa/1000, vpdmin = 0.5, PPFD = metout$SOLR *
    UMOLPERJ * 1, Patm = P_a/1000)
  g_vs_abs_iter <- photo_out$GS
  g_vs_abs_iter[g_vs_abs_iter * f_open < g_vs_abs_iter] <- g_vs_abs_iter[g_vs_abs_iter *
    f_open < g_vs_abs_iter] * f_open[g_vs_abs_iter * f_open <
```



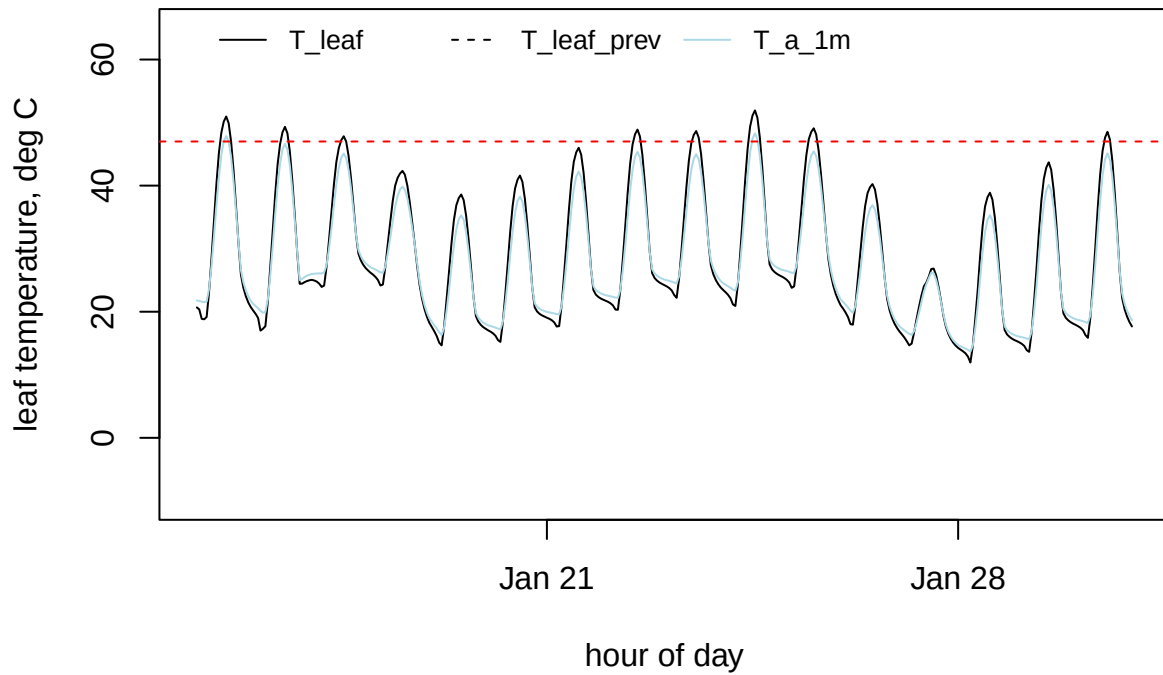
```

g_vs_abs_iter]

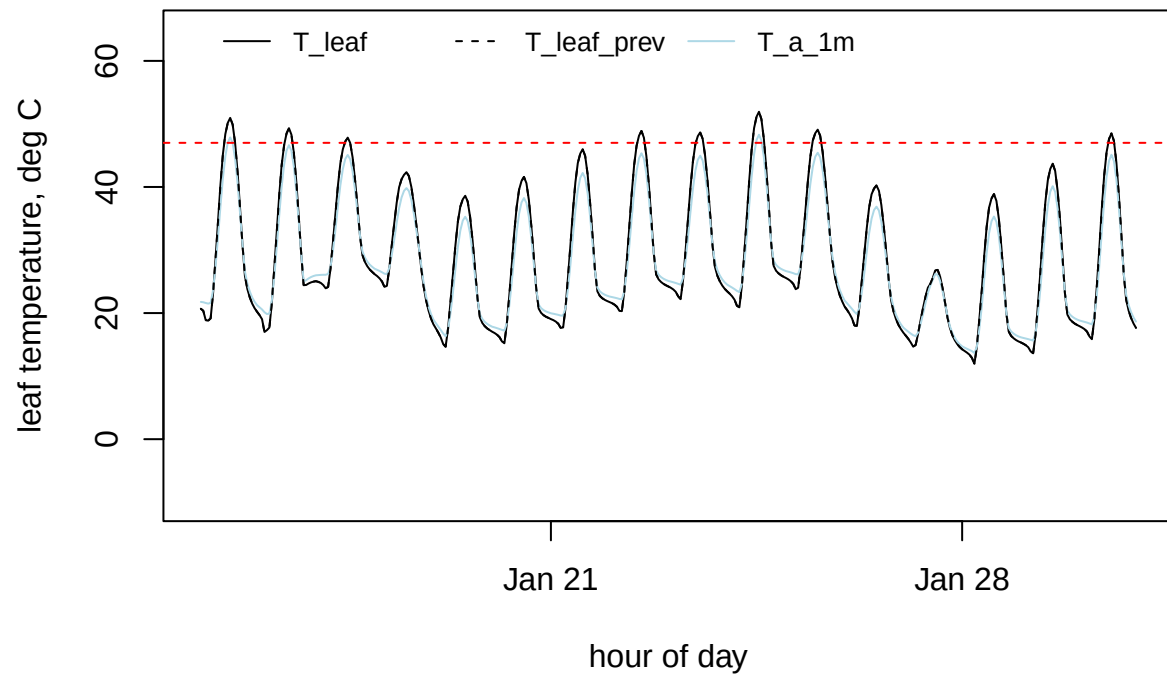
plot(dates[sub], T_leaf_iter[sub], type = "l", col = 1, ylim = c(-10,
  65), main = i, ylab = "leaf temperature, deg C", xlab = "hour of day")
points(dates[sub], new_TAs[sub], type = "l", col = "lightblue")
if (i > 1) {
  points(dates[sub], T_leaf_prev[sub], type = "l", lty = 2)
}
abline(h = 47, col = "red", lty = 2)
legend(dates[sub][1], 68, legend = c("T_leaf", "T_leaf_prev",
  "T_a_1m"), col = c("black", "black", "lightblue"), lty = c(1,
  2, 1), bty = "n", horiz = TRUE, cex = 0.8)
}

```

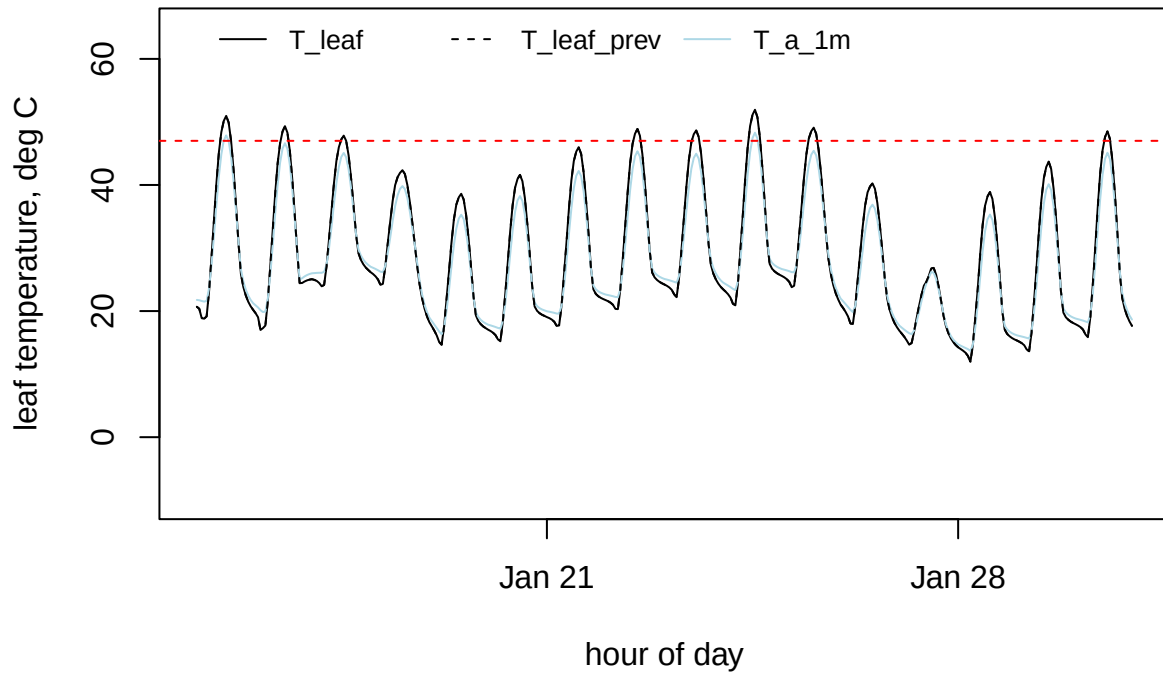
1



2

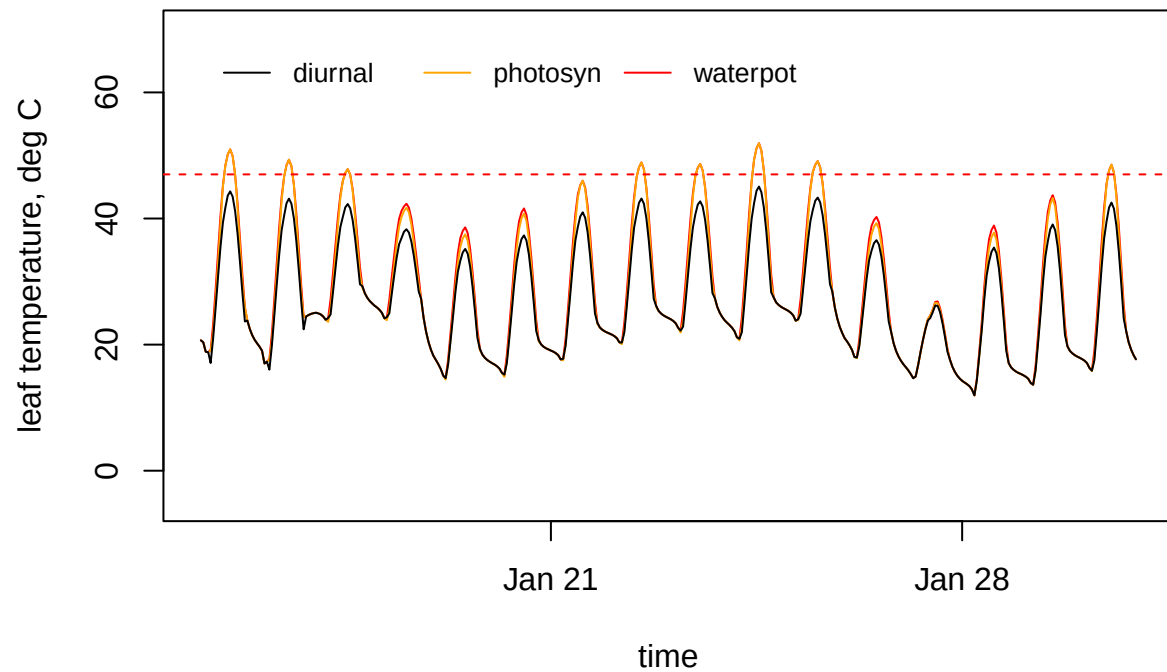


3



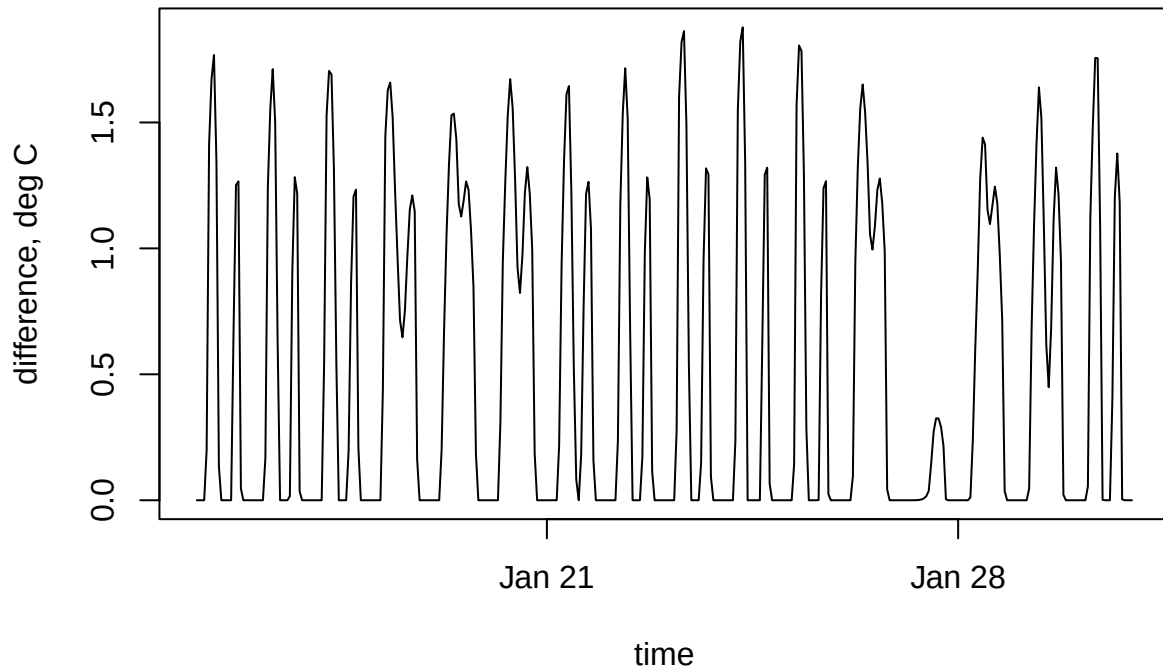
```
T_leaf_pho_pot <- environ$TC # final leaf temperature estimate
```

```
plot(dates[sub], T_leaf_pho_pot[sub], type = "l", ylim = c(-5,
  70), ylab = "leaf temperature, deg C", xlab = "time", col = "red")
points(dates[sub], T_leaf_pho[sub], type = "l", col = "orange")
points(dates[sub], T_leaf[sub], type = "l", col = "black")
legend(dates[sub][1], 68, legend = c("diurnal", "photosyn", "waterpot"),
  col = c("black", "orange", "red"), lty = c(1, 1, 1), bty = "n",
  horiz = TRUE, cex = 0.8)
abline(h = 47, col = "red", lty = 2)
```



You can see that leaves are getting hotter when accounting for stomatal closure due to water stress. Plotting the differences between them reveals that the magnitude of this effect is around 1 or 2 °C.

```
plot(dates[sub], T_leaf_pho_pot[sub] - T_leaf_pho[sub], type = "l",
     ylab = "difference, deg C", xlab = "time")
```



Now recalculate shaded leaf temperature.

```
micro <- micro_highshade_orig
micro$metout <- micro$shadmet # overwrite sun microclimate with shade
micro$soil <- micro$shadsoil # overwrite sun microclimate with shade
dates2 <- micro$dates2 # dates, daily
micro$metout[, 3] <- new_TAs
micro$metout[, 5] <- new_RHs
micro$metout[, 7] <- new_VELs
micro$shadmet[, 3] <- new_TAs_shade
micro$shadmet[, 5] <- new_RHs_shade
micro$shadmet[, 7] <- new_VELs_shade
n <- 3 # iterations to do to find steady state stomatal conductance
# par(mfrow = c(2, 3))
g_vs_abs_iter <- g_vs_abs_pho_pot
T_leaf_iter <- NULL
for (i in 1:n) {
  ecto <- ectotherm(leaf = leaf, live = live, Ww_g = Ww_g,
    shape = shape, shape_b = shape_b, shape_c = shape_c,
    g_vs_ab = g_vs_abs_iter, g_vs_ad = g_vs_ads, g_vs_ab_max = g_vs_ab_max,
    g_vs_ad_max = g_vs_ad_max, epsilon_sub = epsilon_sub,
    epsilon_sky = epsilon_sky, epsilon = epsilon_L, alpha_max = alpha_L,
    alpha_min = alpha_L, M_1 = M_1, fatosk = fatosk, fatosb = fatosb,
    conv_enhance = conv_enhance, pct_cond = pct_cond, postur = postur,
    preshr = P_a, PDIF = PDIFs)
  environ <- as.data.frame(ecto$environ)
```

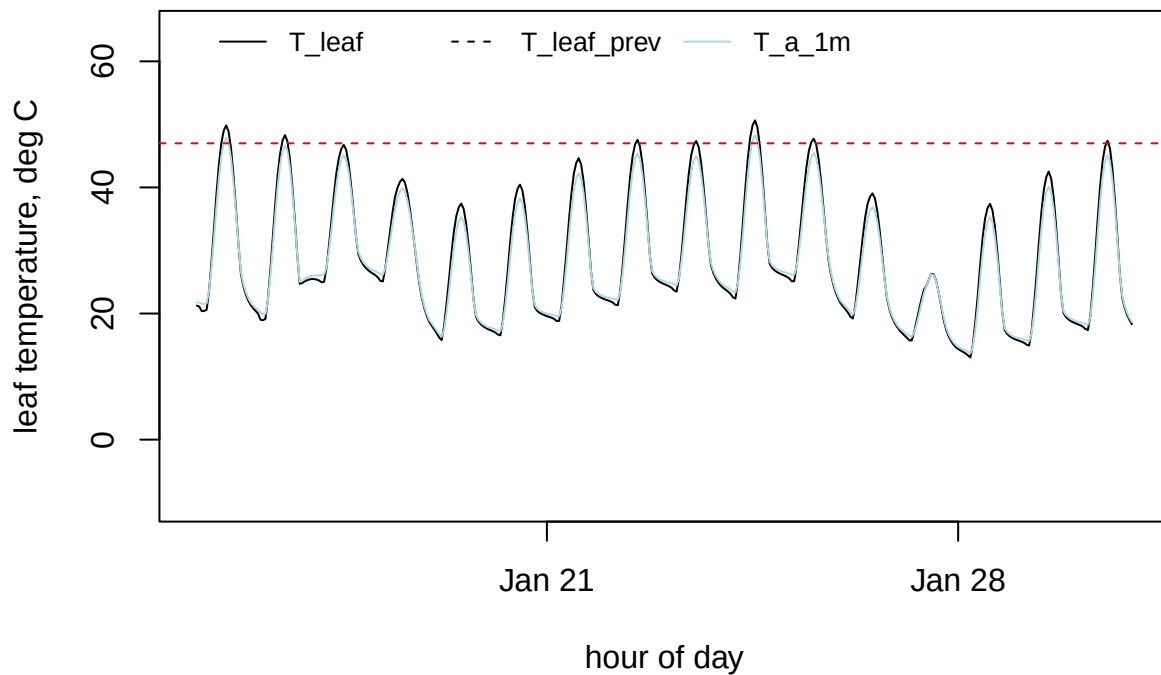
```

if (i > 1) {
  T_leaf_prev <- T_leaf_iter
}
T_leaf_iter <- environ$TC
photo_out <- Photosyn(Tleaf = T_leaf_iter, g1 = 5.2, Ca = 400,
  VPD = VPD_Pa/1000, vpdmin = 0.5, PPFD = metout$SOLR *
  UMOLPERJ * 1, Patm = P_a/1000)
g_vs_abs_iter <- photo_out$GS
g_vs_abs_iter[g_vs_abs_iter * f_open < g_vs_abs_iter] <- g_vs_abs_iter[g_vs_abs_iter *
  f_open < g_vs_abs_iter] * f_open[g_vs_abs_iter * f_open <
  g_vs_abs_iter]

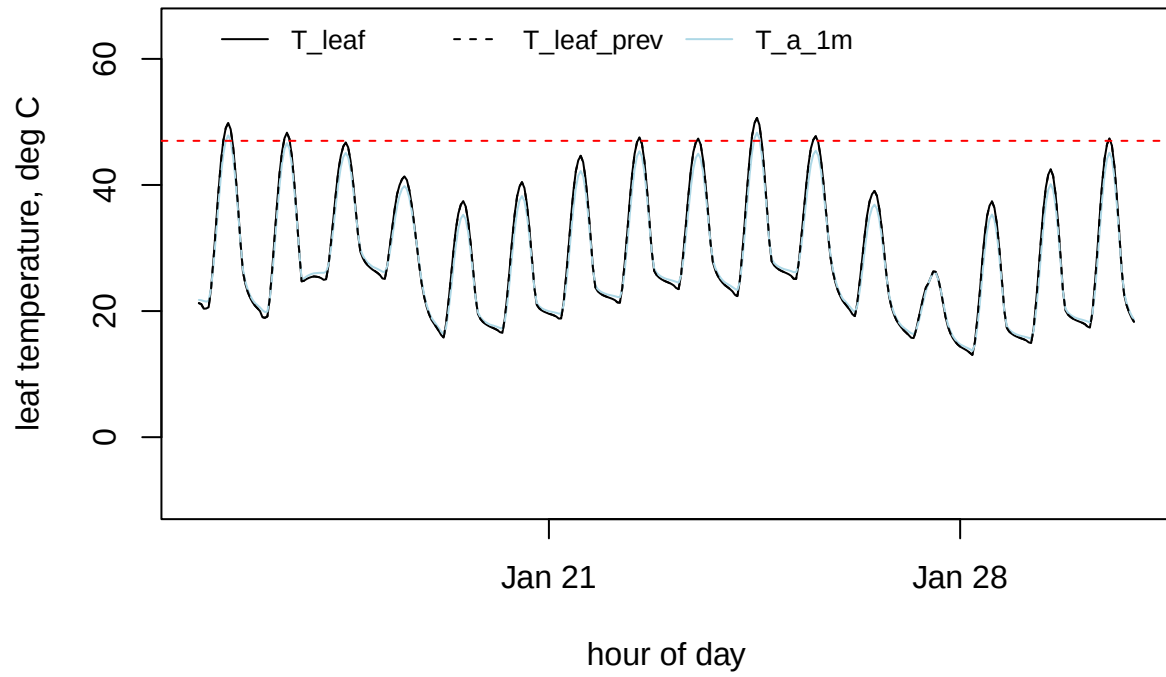
plot(dates[sub], T_leaf_iter[sub], type = "l", col = 1, ylim = c(-10,
  65), main = i, ylab = "leaf temperature, deg C", xlab = "hour of day")
points(dates[sub], new_TAs[sub], type = "l", col = "lightblue")
if (i > 1) {
  points(dates[sub], T_leaf_prev[sub], type = "l", lty = 2)
}
abline(h = 47, col = "red", lty = 2)
legend(dates[sub][1], 68, legend = c("T_leaf", "T_leaf_prev",
  "T_a_1m"), col = c("black", "black", "lightblue"), lty = c(1,
  2, 1), bty = "n", horiz = TRUE, cex = 0.8)
}

```

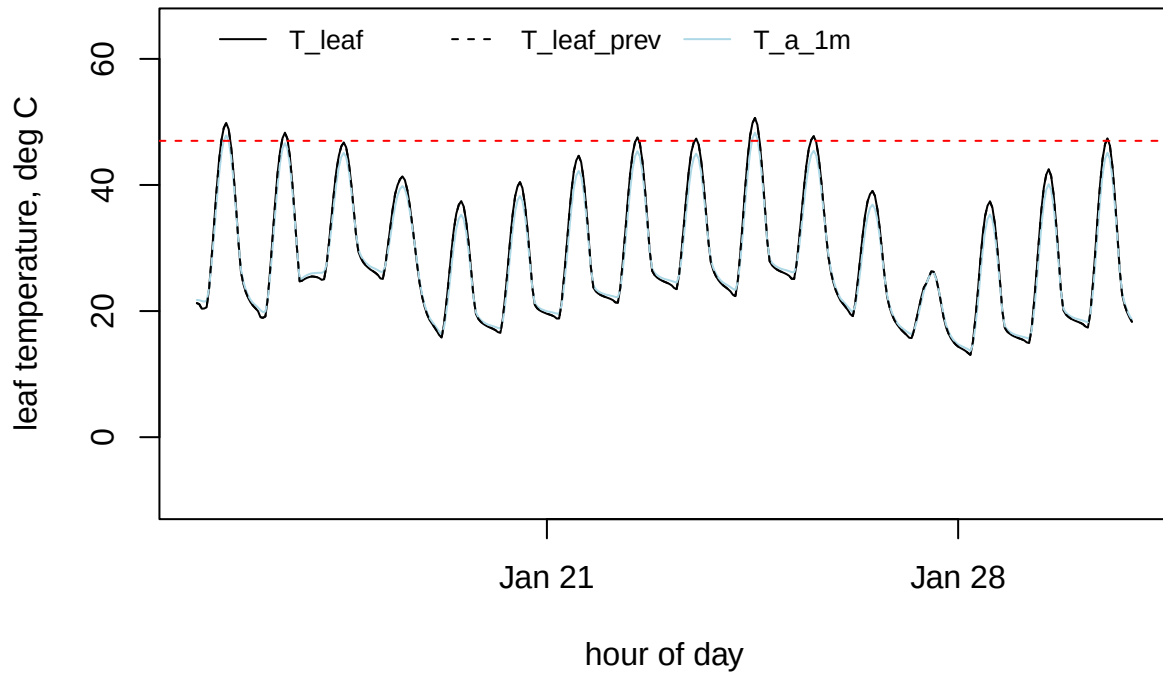
1



2



3



```
T_leaf_pho_pot_shd <- environ$TC # final leaf temperature estimate
```

Now simulate a grasshopper, *Warramaba virgo*, with the DEB model

Load DEB parameters and re-specify thermoregulatory parameters. Start as an egg in the ground at 5 cm.

```
# reading output of parameter estimation in Matlab
library(R.matlab)
```

```
## R.matlab v3.7.0 (2022-08-25 21:52:34 UTC) successfully loaded. See ?R.matlab for help.
```

```
##
## Attaching package: 'R.matlab'
```

```
## The following objects are masked from 'package:base':
##
##      getOption, isOpen
```

```
pars <- readMat("results_Warramaba_virgo.mat")
par.names <- unlist(labels(pars$par))
for (i in 1:length(par.names)) {
  assign(par.names[i], unlist(pars$par[i]))
}
```



```

pars <- readMat("stat_Warramaba_virgo.mat")
par.names <- unlist(labels(pars$stat))
for (i in 1:length(par.names)) {
  assign(par.names[i], unlist(pars$stat[i]))
}
# additional parameters
E_sm <- 2000 # J/cm^2, maximum stomach energy content
fract <- 1 # scaling factor to change size

# initial state for DEB model

# egg
E_init <- (E.0 * fract^4)/(3e-09 * fract^3)
E_H_init <- 0
V_init <- 3e-09
stage <- 0

# hatchling E_init <- E.m E_H_init <- E.Hb V_init <- L.b
# ^ 3 stage <- 1 # adult E_init <- E.m E_H_init <- E.Hp+1
# V_init <- (z ^ 3) * 0.7 stage <- 6

# life history traits
clutchsize <- 8 # eggs per clutch
stages <- 7 # number of stages, 0 = egg, 1-5 = nymphs, 6 = pre egg-laying adult, 7 = post-laying adult
S_instar <- rep(s.1, stages) #stress at instar n: L_n^2/ L_{n-1}^2 (-)}\cr}
L_b <- L.b # structural length at birth (cm)}\cr}
photostart <- 0 # no photoperiod effect on reproduction
photofinish <- 0 # no photoperiod effect on reproduction
reset <- 0 # don't reset lifecycle if death occurs
soilnode <- 3 # depth (soil node) at which eggs laid - 3 = 5 cm

# thermoregulatory parameters
Ww_g <- 0.2 # g, body mass, not used though because DEB model computes this
shape <- 1 # cylinder
shape_b <- 10 # ratio of long to short axis of cylinder
T_pref <- 38 # preferred body temperature (animal will attempt to regulate as close to this value as p
T_F_min <- 13 # degrees C, lower body temperature for foraging
T_F_max <- 40 # degrees C, upper body temperature for foraging
T_B_min <- 13 # degrees C, minimum basking temperature
T_RB_min <- 13 # degrees C, temperature at which animal will move to a basking site
CT_max <- 47 # degrees C, critical thermal maximum (animal will die if CT_kill = 1 and this threshold
CT_min <- 0 # degrees C, critical thermal minimum (used by program to determine depth selected when in

# behavioural traits (made stage-specific later)
diurn <- 1 # diurnal activity allowed (1) or not (0)?
nocturn <- 1 # nocturnal activity allowed (1) or not (0)?
crepus <- 1 # crepuscular activity allowed (1) or not (0)?
burrow <- 0 # shelter in burrow allowed (1) or not (0)?
shade_seek <- 1 # shade seeking allowed (1) or not (0)?
pct_cond <- 0 # Percentage of animal surface contacting the substrate (\%)

```

Now set up microclimate input by ensuring at 1 m height and truncating microclimate so that the simulation

starts on a user-specified oviposition date.

```
# reset to 1m height
micro <- micro_highshade_orig

# micro$shadmet <- micro$metout micro$shadsoil <-
# micro$soil
dates2 <- micro$dates2 # dates, daily
micro$metout[, 3] <- new_TAs
micro$metout[, 5] <- new_RHs
micro$metout[, 7] <- new_VELs
micro$shadmet[, 3] <- new_TAs_shade
micro$shadmet[, 5] <- new_RHs_shade
micro$shadmet[, 7] <- new_VELs_shade
metout <- as.data.frame(micro$metout)
shadmet <- as.data.frame(micro$shadmet)

# make ground and sky temp in shade equal leaf temp
micro$metout[, 14] <- T_leaf_pho_pot
# micro$metout[, 3] <- T_leaf_pho_pot micro$metout[, 4] <-
# T_leaf_pho_pot
micro$soil[, 3] <- T_leaf_pho_pot
micro$shadmet[, 14] <- T_leaf_pho_pot_shd
# micro$shadmet[, 3] <- T_leaf_pho_pot_shd micro$shadmet[,
# 4] <- T_leaf_pho_pot_shd
micro$shadsoil[, 3] <- T_leaf_pho_pot_shd

# set starting date by truncating microclimate model output
oviposition_date <- "2018-03-31 23:00:00"
subs <- which(dates > as.POSIXct(oviposition_date))
subs2 <- which(dates2 > as.POSIXct(oviposition_date))
```

Now run the simulation.

```
ecto <- ectotherm(Ww_g = Ww_g,
  shape = shape,
  shape_b = shape_b,
  T_pref = T_pref,
  T_F_min = T_F_min,
  T_F_max = T_F_max,
  T_B_min = T_B_min,
  T_RB_min = T_RB_min,
  CT_min = CT_min,
  CT_max = CT_max,
  diurn = diurn,
  nocturn = nocturn,
  crepus = crepus,
  shade_seek = shade_seek,
  burrow = burrow,
  pct_cond = pct_cond,
  nyears = ceiling(length(subs2)/365),
  metout = micro$metout[subs,],
  soil = micro$soil[subs,],
  soilpot = micro$soilpot[subs,],
```

```

soilmoist = micro$soilmoist[subs,],
humid = micro$humid[subs,],
tcond = micro$tcond[subs,],
shadmet = micro$shadmet[subs,],
shadsoil = micro$shadsoil[subs,],
shadpot = micro$shadpot[subs,],
shadmoist = micro$shadmoist[subs,],
shadhumid = micro$shadhumid[subs,],
shadtcond = micro$shadtcond[subs,],
rainfall = micro$RAINFALL[subs2],
minshades = micro$minshade[subs2],
maxshades = micro$maxshade[subs2],
DEB = 1, # turns DEB model on
metab_mode = 1, # turns on hemimetabolous model, abp
z = z * fract,
del_M = del.M,
p_Xm = p.Am * 100 / 24 / kap.X,
kap_X = kap.X,
v = v / 24,
kap = kap,
p_M = p.M / 24,
E_G = E.G,
kap_R = kap.R,
k_J = k.J / 24,
E_Hb = E.Hb * fract ^ 3,
E_Hp = (E.Hj+1e-5) * fract ^ 3,
E_Hj = E.Hj * fract ^ 3,
h_a = h.a / (24 ^ 2),
s_G = s.G,
E_0 = E.0 * fract ^ 4,
mu_E = mu.E,
d_V = d.V,
d_E = d.E,
T_REF = T.ref,
T_A = T.A,
T_AL = T.AL,
T_AH = T.AH,
T_L = T.L,
T_H = T.H,
E_sm = E_sm,
E_init = E_init,
E_H_init = E_H_init,
V_init = V_init,
stage = stage,
clutchsize = clutchsize,
stages = stages,
S_instar = rep(s.1, stages),
L_b = L.b,
photostart = photostart,
photofinish = photofinish,
reset = reset,
soilnode = soilnode)

```

```
## running ectotherm model ...
```

```
## runtime 14.56 seconds
```

```
datetime <- dates[subs]
environ <- as.data.frame(ecto$environ)
enbal <- as.data.frame(ecto$enbal)
environ <- cbind(datetime, environ)
enbal <- cbind(datetime, enbal)
debout <- as.data.frame(ecto$debout)
debout <- cbind(datetime, debout)
```

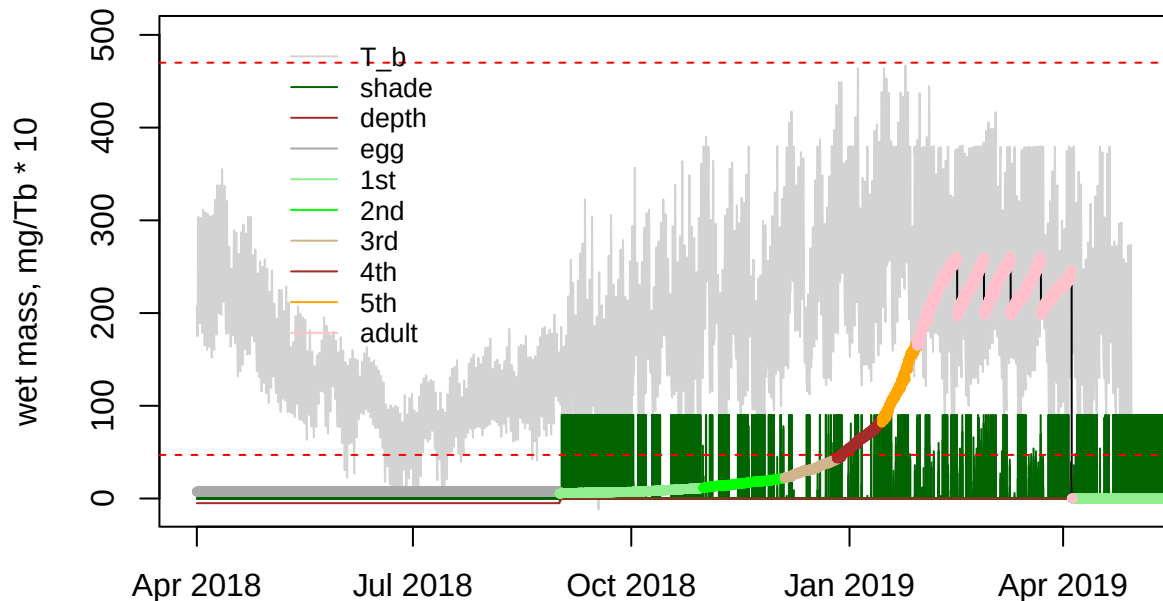
Plot the life cycle compared to temperature.

```
sub <- which(datetime > as.POSIXct("2018-03-31") & datetime <
  as.POSIXct("2019-04-30"))
with(environ[sub, ], {
  plot(TC * 10 ~ datetime, type = "l", xlab = "", ylab = "wet mass, mg/Tb * 10",
    col = "lightgrey", ylim = c(-10, 500))
})
with(environ, {
  points(SHADE ~ datetime, type = "h", col = "darkgreen", ylim = c(0,
    300))
})
with(environ, {
  points(DEP ~ datetime, type = "l", col = "brown", ylim = c(0,
    300))
})
with(debout, {
  points(WETMASS * 1000 ~ datetime, type = "l", col = "black",
    ylim = c(0, 300))
})
with(subset(debout, STAGE == 0), {
  points(WETMASS * 1000 ~ datetime, type = "p", lwd = 2, cex = 0.4,
    col = "darkgrey")
})
with(subset(debout, STAGE == 1), {
  points(WETMASS * 1000 ~ datetime, type = "p", lwd = 2, cex = 0.4,
    col = "lightgreen")
})
with(subset(debout, STAGE == 2), {
  points(WETMASS * 1000 ~ datetime, type = "p", lwd = 2, cex = 0.4,
    col = "green")
})
with(subset(debout, STAGE == 3), {
  points(WETMASS * 1000 ~ datetime, type = "p", lwd = 2, cex = 0.4,
    col = "tan")
})
with(subset(debout, STAGE == 4), {
  points(WETMASS * 1000 ~ datetime, type = "p", lwd = 2, cex = 0.4,
    col = "brown")
})
with(subset(debout, STAGE == 5), {
```

```

points(WETMASS * 1000 ~ datetime, type = "p", lwd = 2, cex = 0.4,
       col = "orange")
})
with(subset(debout, STAGE == 6), {
  points(WETMASS * 1000 ~ datetime, type = "p", lwd = 2, cex = 0.4,
        col = "pink")
})
abline(h = 47 * 10, lty = 2, col = "red")
legend(enviro[sub, ]$datetime[1] + 3600 * 30 * 24, 510, legend = c("T_b",
  "shade", "depth", "egg", "1st", "2nd", "3rd", "4th", "5th",
  "adult"), col = c("lightgrey", "darkgreen", "brown", "darkgrey",
  "lightgreen", "green", "tan", "brown", "orange", "pink"),
  lty = 1, bty = "n", horiz = FALSE, cex = 0.8)
abline(h = 47, col = "red", lty = 2)

```



Zoom in on heat wave.

```

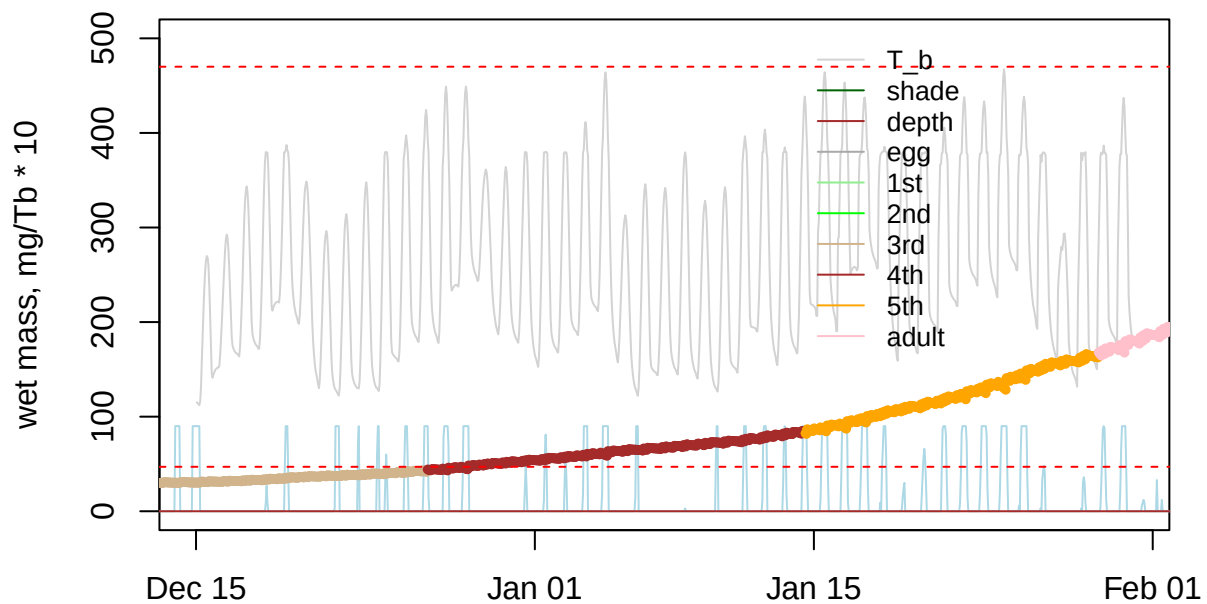
sub <- which(datetime > as.POSIXct("2018-12-15") & datetime <
  as.POSIXct("2019-01-31"))
# sub <- which(datetime > as.POSIXct('2019-01-01') &
# datetime < as.POSIXct('2019-01-10'))
with(enviro[sub, ], {
  plot(TC * 10 ~ datetime, type = "l", xlab = "", ylab = "wet mass, mg/Tb * 10",
       col = "lightgrey", ylim = c(0, 500))
})
with(enviro, {

```

```

    points(SHADE ~ datetime, type = "l", col = "lightblue", ylim = c(0,
      300))
  })
  # points(dates, metout$TAREF*10, col = 'blue', type = 'l')
  # with(enviro, {points(TA*10 ~ datetime, type = 'l', col =
  # 'lightblue', ylim = c(0, 300))})
  with(enviro, {
    points(DEP ~ datetime, type = "l", col = "brown", ylim = c(0,
      300))
  })
  with(debout, {
    points(WETMASS * 1000 ~ datetime, type = "l", col = "black",
      ylim = c(0, 300))
  })
  with(subset(debout, STAGE == 0), {
    points(WETMASS * 1000 ~ datetime, type = "p", lwd = 2, cex = 0.4,
      col = "darkgrey")
  })
  with(subset(debout, STAGE == 1), {
    points(WETMASS * 1000 ~ datetime, type = "p", lwd = 2, cex = 0.4,
      col = "lightgreen")
  })
  with(subset(debout, STAGE == 2), {
    points(WETMASS * 1000 ~ datetime, type = "p", lwd = 2, cex = 0.4,
      col = "green")
  })
  with(subset(debout, STAGE == 3), {
    points(WETMASS * 1000 ~ datetime, type = "p", lwd = 2, cex = 0.4,
      col = "tan")
  })
  with(subset(debout, STAGE == 4), {
    points(WETMASS * 1000 ~ datetime, type = "p", lwd = 2, cex = 0.4,
      col = "brown")
  })
  with(subset(debout, STAGE == 5), {
    points(WETMASS * 1000 ~ datetime, type = "p", lwd = 2, cex = 0.4,
      col = "orange")
  })
  with(subset(debout, STAGE == 6), {
    points(WETMASS * 1000 ~ datetime, type = "p", lwd = 2, cex = 0.4,
      col = "pink")
  })
  abline(h = 47 * 10, lty = 2, col = "red")
  legend(enviro[sub, ]$datetime[1] + 3600 * 30 * 24, 510, legend = c("T_b",
    "shade", "depth", "egg", "1st", "2nd", "3rd", "4th", "5th",
    "adult"), col = c("lightgrey", "darkgreen", "brown", "darkgrey",
    "lightgreen", "green", "tan", "brown", "orange", "pink"),
    lty = 1, bty = "n", horiz = FALSE, cex = 0.8)
  abline(h = 47, col = "red", lty = 2)

```



Plot activity window

```
sub <- which(datetime > as.POSIXct("2018-03-31") & datetime <
  as.POSIXct("2019-04-30"))
# activity plot
environ2 <- environ
environ2$ACT[environ2$ACT != 2] <- 0
environ2$ACT[environ2$ACT == 2] <- 1

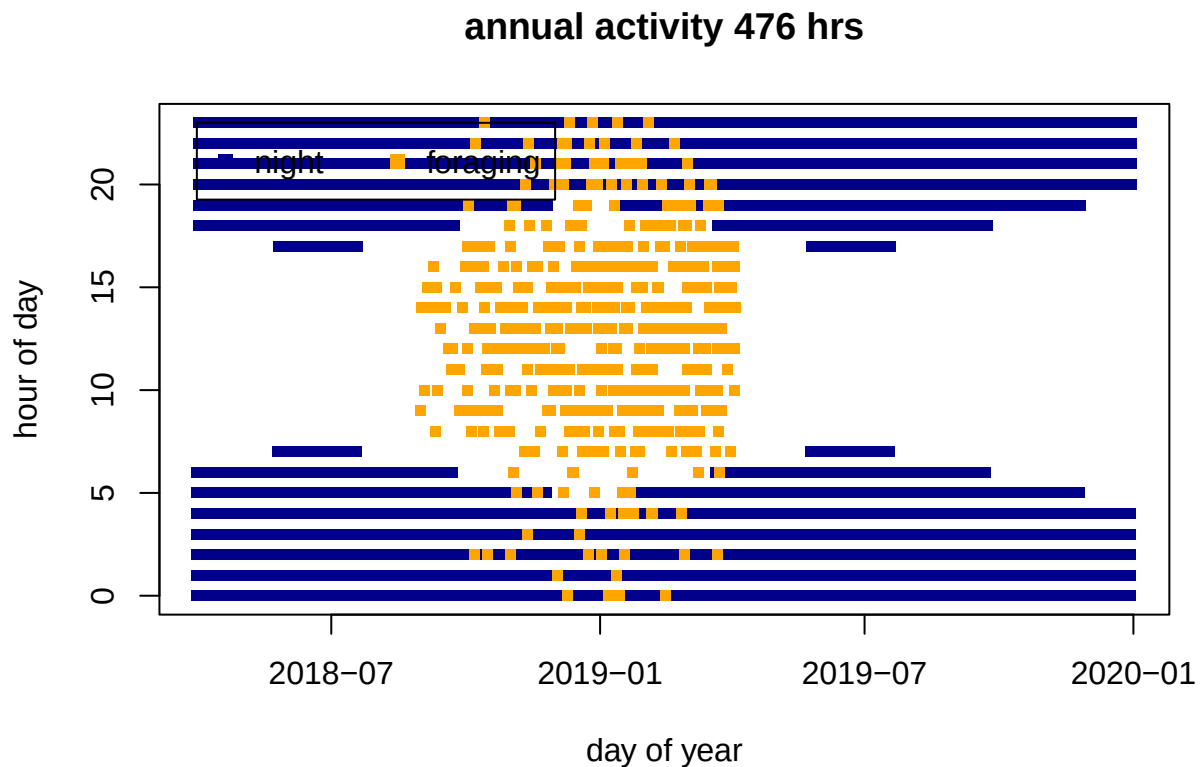
acthrs <- aggregate(environ2$ACT, by = list(environ2$DOY), FUN = sum)
yearact <- sum(acthrs$x)

# seasonal activity plot (dark blue = night, light blue =
# basking, orange = foraging)
forage <- subset(environ, ACT == 2)
bask <- subset(environ, ACT == 1)
night <- subset(environ, ZEN == 90)

with(night[sub, ], plot(as.Date(datetime), TIME, ylab = "hour of day",
  xlab = "day of year", pch = 15, cex = 0.75, col = "dark blue",
  main = paste0("annual activity ", yearact, " hrs")))
# nighttime hours
with(forage, points(TIME ~ as.Date(datetime), pch = 15, cex = 0.75,
  col = "orange")) # foraging Tbs

legend(as.Date(night[sub, ]$datetime[1]), 23, c("night", "foraging"),
```

```
pch = 15, cex = 1, col = c("dark blue", "orange"), horiz = TRUE)
```



Add code to calculate heat stress via death time curve approach with some made up parameters.

Now simulate a dragon lizard, *Ctenophorus ornatus*, with the DEB model

Load DEB parameters and re-specify thermoregulatory parameters. Start as an egg in the ground at 10 cm.

```
species <- "Ctenophorus.ornatus"
# loading parameters from allStat
load("allstat.Rda")
allDEB.species <- unlist(labels(allStat$allStat))
if (exists("E.Hj") == TRUE) {
  rm(E.Hj)
} # remove previous value from memory if present from prior run
if (exists("L.j") == TRUE) {
  rm(L.j)
} # remove previous value from memory if present from prior run
species.slot <- which(allDEB.species == species) # find the slot with the species of interest
par.names <- unlist(labels(allStat$allStat[[species.slot]])) # get parameter names
for (i in 1:length(par.names)) {
  # pull parameters into memory
  assign(par.names[i], unlist(allStat$allStat[[species.slot]][i]))
}
rm(allStat)
```



```

# additional parameters
E_sm <- 1000 # J/cm^2, maximum stomach energy content
fract <- 1 # scaling factor to change size

# initial state for DEB model

# egg
E_init <- (E.0 * fract^4)/(3e-09 * fract^3)
E_H_init <- 0
V_init <- 3e-09
stage <- 0

# hatchling E_init <- E.m E_H_init <- E.Hb V_init <- L.b
# ^ 3 stage <- 1 # adult E_init <- E.m E_H_init <- E.Hp+1
# V_init <- (z ^ 3) * 0.7 stage <- 6

# life history traits
clutchsize <- 4 # eggs per clutch
stages <- 3 # number of stages, 0 = egg, 1-5 = nymphs, 6 = pre egg-laying adult, 7 = post-laying adult
photostart <- 3 # no photoperiod effect on reproduction
photofinish <- 1 # no photoperiod effect on reproduction
reset <- 0 # don't reset lifecycle if death occurs
soilnode <- 3 # depth (soil node) at which eggs laid - 3 = 5 cm

# thermoregulatory parameters
Ww_g <- 40 # g, body mass
shape <- 4 # lizard
T_pref <- 41 # preferred body temperature (animal will attempt to regulate as close to this value as p
T_F_min <- 30 # degrees C, lower body temperature for foraging
T_F_max <- 41 # degrees C, upper body temperature for foraging
T_B_min <- 20 # degrees C, minimum basking temperature
T_RB_min <- 13 # degrees C, temperature at which animal will move to a basking site
CT_max <- 47 # degrees C, critical thermal maximum (animal will die if CT_kill = 1 and this threshold
CT_min <- 0 # degrees C, critical thermal minimum (used by program to determine depth selected when in

# behavioural traits (made stage-specific later)
diurn <- 1 # diurnal activity allowed (1) or not (0)?
nocturn <- 0 # nocturnal activity allowed (1) or not (0)?
crepus <- 0 # crepuscular activity allowed (1) or not (0)?
shade_seek <- 1 # shade seeking allowed (1) or not (0)?
burrow <- 1 # shelter in burrow allowed (1) or not (0)?
pct_cond <- 10 # Percentage of animal surface contacting the substrate (\%)

# reset to 1m height
micro <- micro_highshade_orig
metout <- as.data.frame(micro$metout)
shadmet <- as.data.frame(micro$shadmet)

# make ground and sky temp in shade equal leaf temp
micro$metout[, 14] <- T_leaf_pho_pot
# micro$metout[, 3] <- T_leaf_pho_pot micro$metout[, 4] <-
# T_leaf_pho_pot micro$soil[, 3] <- T_leaf_pho_pot

```

```

micro$shadmet[, 14] <- T_leaf_pho_pot_shd
# micro$shadmet[, 3] <- T_leaf_pho_pot_shd micro$shadmet[,
# 4] <- T_leaf_pho_pot_shd micro$shadsoil[, 3] <-
# T_leaf_pho_pot_shd

# set starting date by truncating microclimate model output
oviposition_date <- "2018-02-01 23:00:00"
subs <- which(dates > as.POSIXct(oviposition_date))
subs2 <- which(dates2 > as.POSIXct(oviposition_date))

```

Now run the simulation.

```

ecto <- ectotherm(Ww_g = Ww_g,
  shape = shape,
  shape_b = shape_b,
  T_pref = T_pref,
  T_F_min = T_F_min,
  T_F_max = T_F_max,
  T_B_min = T_B_min,
  T_RB_min = T_RB_min,
  CT_min = CT_min,
  CT_max = CT_max,
  diurn = diurn,
  nocturn = nocturn,
  crepus = crepus,
  shade_seek = shade_seek,
  burrow = burrow,
  pct_cond = pct_cond,
  nyears = ceiling(length(subs2)/365),
  metout = micro$metout[subs,],
  soil = micro$soil[subs,],
  soilpot = micro$soilpot[subs,],
  soilmoist = micro$soilmoist[subs,],
  humid = micro$humid[subs,],
  tcond = micro$tcond[subs,],
  shadmet = micro$shadmet[subs,],
  shadsoil = micro$shadsoil[subs,],
  shadpot = micro$shadpot[subs,],
  shadmoist = micro$shadmoist[subs,],
  shadhumid = micro$shadhumid[subs,],
  shadtcond = micro$shadtcond[subs,],
  rainfall = micro$RAINFALL[subs2],
  minshades = micro$minshade[subs2],
  maxshades = micro$maxshade[subs2],
  DEB = 1, # turns DEB model on
  metab_mode = 0, # not an insect
  X = 0.03,
  z = z * fract,
  del_M = del.M,
  p_Xm = p.Am * 100 / 24 / kap.X,
  kap_X = kap.X,
  v = v / 24,
  kap = kap,

```

```

p_M = p.M / 24,
E_G = E.G,
kap_R = kap.R,
k_J = k.J / 24,
E_Hb = E.Hb * fract ^ 3,
E_Hp = E.Hp * fract ^ 3,
h_a = h.a / (24 ^ 2),
s_G = s.G,
E_O = E.O * fract ^ 4,
mu_E = mu.E,
d_V = d.V,
d_E = d.E,
T_REF = T.ref,
T_A = T.A,
T_AL = T.AL,
T_AH = T.AH,
T_L = T.L,
T_H = T.H,
E_sm = E.sm,
E_init = E.init,
E_H_init = E_H.init,
V_init = V.init,
stage = stage,
clutchsize = clutchsize,
stages = stages,
photostart = photostart,
photofinish = photofinish,
reset = reset,
soilnode = soilnode)

```

```
## running ectotherm model ...
```

```
## runtime 4.75 seconds
```

```

datetime <- dates[subs]
environ <- as.data.frame(ecto$environ)
enbal <- as.data.frame(ecto$enbal)
environ <- cbind(datetime, environ)
enbal <- cbind(datetime, enbal)
debout <- as.data.frame(ecto$debout)
debout <- cbind(datetime, debout)

```

Plot the life cycle compared to temperature.

```

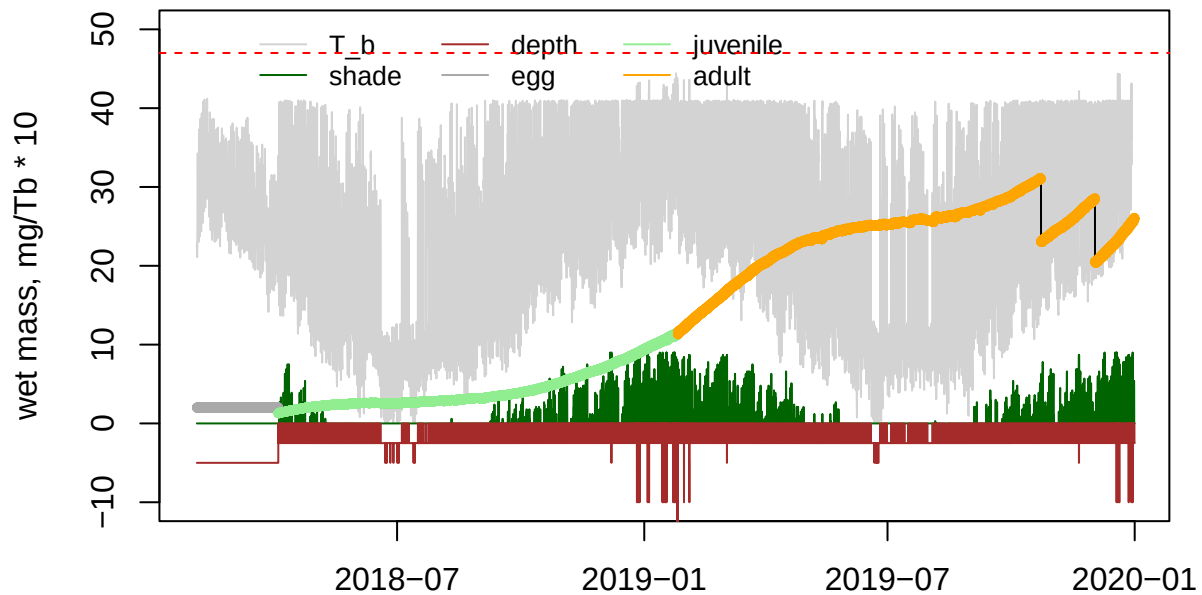
sub <- which(datetime > as.POSIXct("2018-01-31") & datetime <
  as.POSIXct("2019-12-30"))
with(environ[sub, ], {
  plot(TC ~ datetime, type = "l", xlab = "", ylab = "wet mass, mg/Tb * 10",
    col = "lightgrey", ylim = c(-10, 50))
})
abline(h = 47, lty = 2, col = "red")
with(environ, {

```

```

    points(SHADE/10 ~ datetime, type = "l", col = "darkgreen",
           ylim = c(0, 300))
  })
  with(envIRON, {
    points(DEP ~ datetime, type = "l", col = "brown", ylim = c(0,
    300))
  })
  with(debout, {
    points(WETMASS ~ datetime, type = "l", col = "black", ylim = c(0,
    300))
  })
  with(subset(debout, STAGE == 0), {
    points(WETMASS ~ datetime, type = "p", lwd = 2, cex = 0.4,
           col = "darkgrey")
  })
  with(subset(debout, STAGE == 1), {
    points(WETMASS ~ datetime, type = "p", lwd = 2, cex = 0.4,
           col = "lightgreen")
  })
  with(subset(debout, STAGE == 2), {
    points(WETMASS ~ datetime, type = "p", lwd = 2, cex = 0.4,
           col = "orange")
  })
  legend(envIRON[sub, ]$datetime[1] + 3600 * 30 * 24, 52, legend = c("T_b",
    "shade", "depth", "egg", "juvenile", "adult"), col = c("lightgrey",
    "darkgreen", "brown", "darkgrey", "lightgreen", "orange"),
    lty = 1, bty = "n", horiz = FALSE, cex = 0.8, ncol = 3)
  abline(h = 47, col = "red", lty = 2)

```



Plot activity window

```
# activity plot
environ2 <- environ
environ2$ACT[environ2$ACT != 2] <- 0
environ2$ACT[environ2$ACT == 2] <- 1

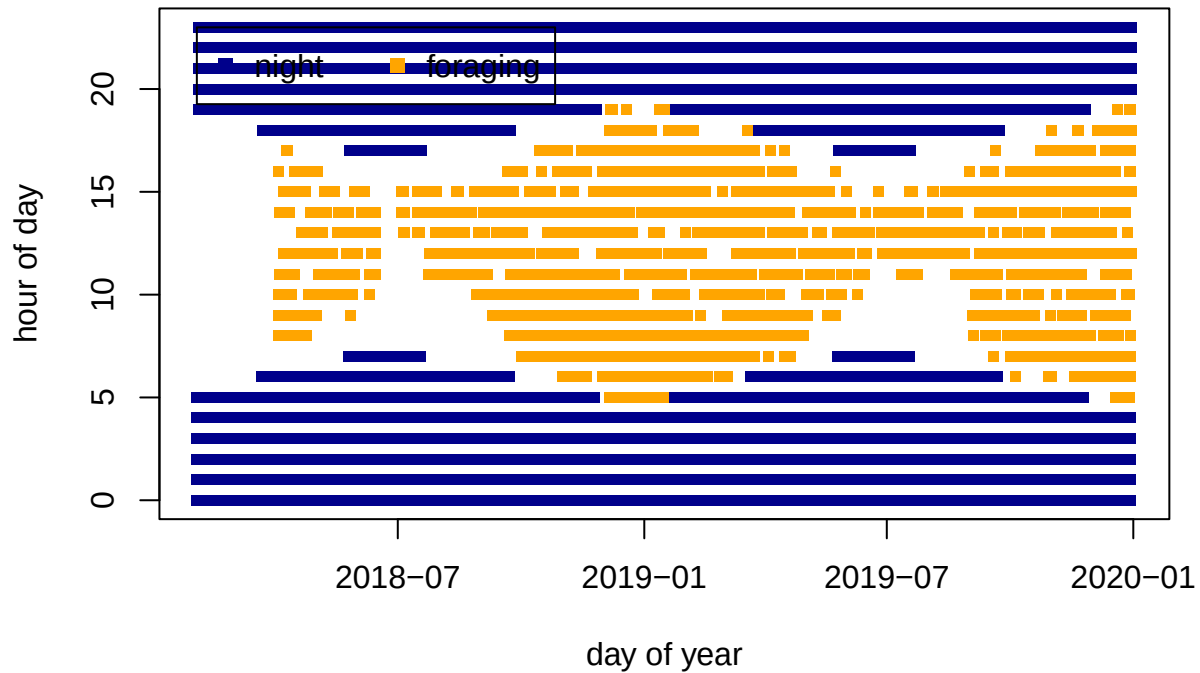
acthrs <- aggregate(environ2$ACT, by = list(environ2$DOY), FUN = sum)
yearact <- sum(acthrs$x)

# seasonal activity plot (dark blue = night, light blue =
# basking, orange = foraging)
forage <- subset(environ, ACT == 2)
bask <- subset(environ, ACT == 1)
night <- subset(environ, ZEN == 90)

with(night[sub, ], plot(as.Date(datetime), TIME, ylab = "hour of day",
  xlab = "day of year", pch = 15, cex = 0.75, col = "dark blue",
  main = paste0("annual activity ", yearact, " hrs")))
# nighttime hours
with(forage, points(TIME ~ as.Date(datetime), pch = 15, cex = 0.75,
  col = "orange")) # foraging Tbs

legend(as.Date(night[sub, ]$datetime[1]), 23, c("night", "foraging"),
  pch = 15, cex = 1, col = c("dark blue", "orange"), horiz = TRUE)
```

annual activity 2018 hrs



Now do it all again but with low shade - e.g. because of vegetation die-off.

```
# reset to 1m height
micro <- micro_lowshade_orig
metout <- as.data.frame(micro$metout)
shadmet <- as.data.frame(micro$shadmet)

# make ground and sky temp in shade equal leaf temp
micro$metout[, 14] <- T_leaf_pho_pot
# micro$metout[, 3] <- T_leaf_pho_pot micro$metout[, 4] <-
# T_leaf_pho_pot micro$soil[, 3] <- T_leaf_pho_pot
micro$shadmet[, 14] <- T_leaf_pho_pot_shd
# micro$shadmet[, 3] <- T_leaf_pho_pot_shd micro$shadmet[,
# 4] <- T_leaf_pho_pot_shd micro$shadsoil[, 3] <-
# T_leaf_pho_pot_shd

# set starting date by truncating microclimate model output
oviposition_date <- "2018-02-01 23:00:00"
subs <- which(dates > as.POSIXct(oviposition_date))
subs2 <- which(dates2 > as.POSIXct(oviposition_date))
```

Now run the simulation.

```
ecto <- ectotherm(Ww_g = Ww_g,
                  shape = shape,
                  shape_b = shape_b,
```

```

T_pref = T_pref,
T_F_min = T_F_min,
T_F_max = T_F_max,
T_B_min = T_B_min,
T_RB_min = T_RB_min,
CT_min = CT_min,
CT_max = CT_max,
diurn = diurn,
nocturn = nocturn,
crepus = crepus,
shade_seek = shade_seek,
burrow = burrow,
pct_cond = pct_cond,
years = ceiling(length(subs2)/365),
metout = micro$metout[sub],
soil = micro$soil[sub],
soilpot = micro$soilpot[sub],
soilmoist = micro$soilmoist[sub],
humid = micro$humid[sub],
tcond = micro$tcond[sub],
shadmet = micro$shadmet[sub],
shadsoil = micro$shadsoil[sub],
shadpot = micro$shadpot[sub],
shadmoist = micro$shadmoist[sub],
shadhumid = micro$shadhumid[sub],
shadtcond = micro$shadtcond[sub],
rainfall = micro$RAINFALL[sub],
minshades = micro$minshade[sub],
maxshades = micro$maxshade[sub],
DEB = 1, # turns DEB model on
metab_mode = 0, # not an insect
X = 0.03,
z = z * fract,
del_M = del.M,
p_Xm = p.Am * 100 / 24 / kap.X,
kap_X = kap.X,
v = v / 24,
kap = kap,
p_M = p.M / 24,
E_G = E.G,
kap_R = kap.R,
k_J = k.J / 24,
E_Hb = E.Hb * fract ^ 3,
E_Hp = E.Hp * fract ^ 3,
h_a = h.a / (24 ^ 2),
s_G = s.G,
E_0 = E.0 * fract ^ 4,
mu_E = mu.E,
d_V = d.V,
d_E = d.E,
T_REF = T.ref,
T_A = T.A,
T_AL = T.AL,

```

```

T_AH = T.AH,
T_L = T.L,
T_H = T.H,
E_sm = E_sm,
E_init = E_init,
E_H_init = E_H_init,
V_init = V_init,
stage = stage,
clutchsize = clutchsize,
stages = stages,
photostart = photostart,
photofinish = photofinish,
reset = reset,
soilnode = soilnode)

```

```
## running ectotherm model ...
```

```
## runtime 2.97 seconds
```

```

datetime <- dates[subs]
environ <- as.data.frame(ecto$environ)
enbal <- as.data.frame(ecto$enbal)
environ <- cbind(datetime, environ)
enbal <- cbind(datetime, enbal)
debout <- as.data.frame(ecto$debout)
debout <- cbind(datetime, debut)

```

Plot the life cycle compared to temperature.

```

sub <- which(datetime > as.POSIXct("2018-01-31") & datetime <
  as.POSIXct("2019-12-30"))
with(environ[sub, ], {
  plot(TC ~ datetime, type = "l", xlab = "", ylab = "wet mass, mg/Tb * 10",
    col = "lightgrey", ylim = c(-10, 50))
})
abline(h = 47, lty = 2, col = "red")
with(environ, {
  points(SHADE/10 ~ datetime, type = "l", col = "darkgreen",
    ylim = c(0, 300))
})
with(environ, {
  points(DEP ~ datetime, type = "l", col = "brown", ylim = c(0,
    300))
})
with(debout, {
  points(WETMASS ~ datetime, type = "l", col = "black", ylim = c(0,
    300))
})
with(subset(debout, STAGE == 0), {
  points(WETMASS ~ datetime, type = "p", lwd = 2, cex = 0.4,
    col = "darkgrey")
})

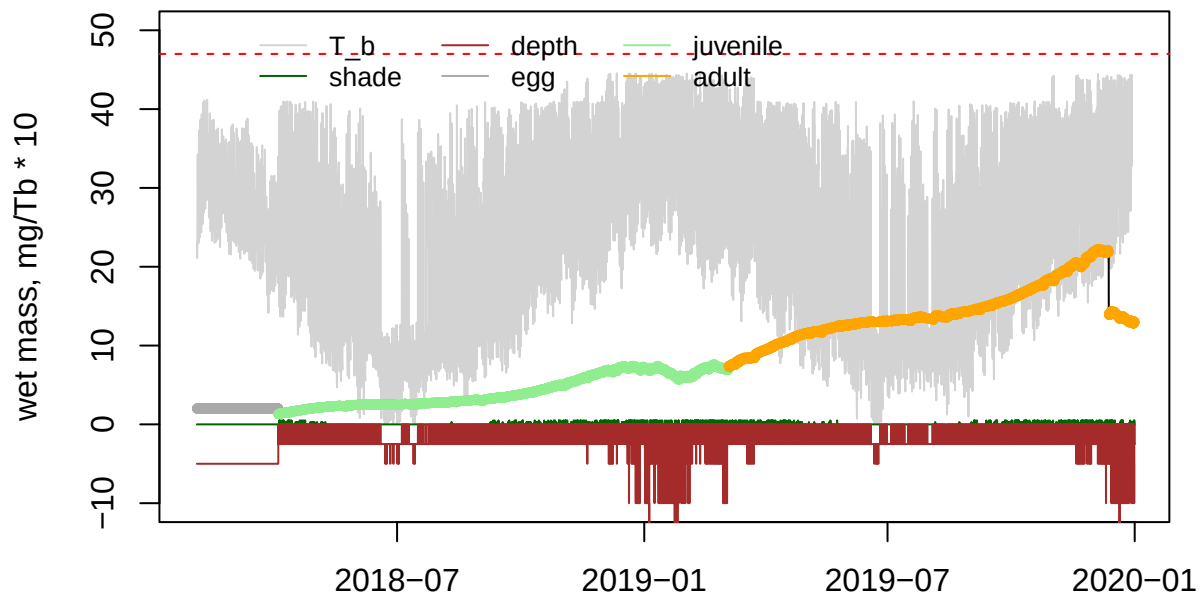
```



```

with(subset(debout, STAGE == 1), {
  points(WETMASS ~ datetime, type = "p", lwd = 2, cex = 0.4,
    col = "lightgreen")
})
with(subset(debout, STAGE == 2), {
  points(WETMASS ~ datetime, type = "p", lwd = 2, cex = 0.4,
    col = "orange")
})
legend(enviro[sub, ]$datetime[1] + 3600 * 30 * 24, 52, legend = c("T_b",
  "shade", "depth", "egg", "juvenile", "adult"), col = c("lightgrey",
  "darkgreen", "brown", "darkgrey", "lightgreen", "orange"),
  lty = 1, bty = "n", horiz = FALSE, cex = 0.8, ncol = 3)

```



Plot activity window

```

# activity plot
environ2 <- environ
environ2$ACT[environ2$ACT != 2] <- 0
environ2$ACT[environ2$ACT == 2] <- 1

acthrs <- aggregate(environ2$ACT, by = list(environ2$DOY), FUN = sum)
yearact <- sum(acthrs$x)

# seasonal activity plot (dark blue = night, light blue =
# basking, orange = foraging)
forage <- subset(environ, ACT == 2)

```

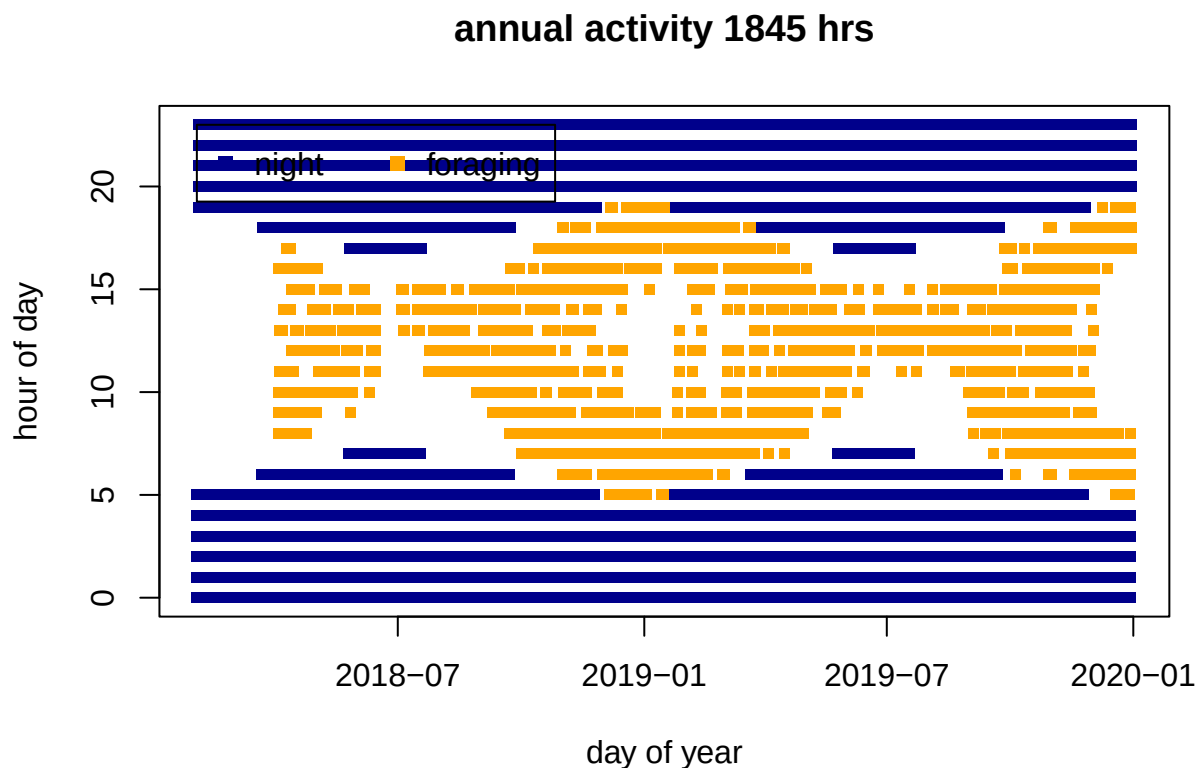
```

bask <- subset(envIRON, ACT == 1)
night <- subset(envIRON, ZEN == 90)

with(night[sub, ], plot(as.Date(datetime), TIME, ylab = "hour of day",
  xlab = "day of year", pch = 15, cex = 0.75, col = "dark blue",
  main = paste0("annual activity ", yearact, " hrs")))
# nighttime hours
with(forage, points(TIME ~ as.Date(datetime), pch = 15, cex = 0.75,
  col = "orange")) # foraging Tbs

legend(as.Date(night[sub, ]$datetime[1]), 23, c("night", "foraging"),
  pch = 15, cex = 1, col = c("dark blue", "orange"), horiz = TRUE)

```



References

- Abatzoglou, J. T. 2011. Development of gridded surface meteorological data for ecological applications and modelling. *International Journal of Climatology* 33:121–131.
- Campbell, G. S., and J. M. Norman. 1998. *Environmental Biophysics*. Springer, New York.
- Duursma, R. A. 2015. Plantecophys - An R Package for Analysing and Modelling Leaf Gas Exchange Data. *PLOS ONE* 10:e0143346.
- Kearney, M. R., and W. P. Porter. 2020. NicheMapR – an R package for biophysical modelling: the ectotherm and Dynamic Energy Budget models. *Ecography* 43:85–96.

Klinges, D. H., J. P. Duffy, M. R. Kearney, and I. M. D. Maclean. 2022. mcera5: Driving microclimate models with ERA5 global gridded climate data. *Methods in Ecology and Evolution* 13:7.

Maclean, I. M. D., J. R. Mosedale, and J. J. Bennie. 2019. Microclima: An r package for modelling meso- and microclimate. *Methods in Ecology and Evolution* 10:280–290.

Muir, C. D. 2019. tealeaves: an R package for modelling leaf temperature using energy budgets. *AoB PLANTS* 11:plz054.